Web Programming

Rodrigo Martins Pagliares

Last update: 10/27/2025

Web Programming

Rodrigo Martins Pagliares

JavaScript
Dynamic client-side scripting

JavaScript first steps

developer.mozilla.org

Get real-time assistance with your coding queries. Try AI Help now!

mdn web docs _ References Guides Plus Blog Play NEW AI Help BETA Theme Log in Get MDN Plus

Guides English (US)

Filter

HTML — Structuring the web

- Introduction to HTML
- Multimedia and embedding
- HTML tables

CSS — Styling the web

- CSS first steps
- CSS building blocks
- Styling text
- CSS layout

JavaScript — Dynamic client-side scripting

- JavaScript first steps
- JavaScript building blocks
- Introducing JavaScript objects
- Asynchronous JavaScript
- Client-side web APIs

Web forms — Working with user data

Learn web development

Welcome to the MDN learning area. This set of articles aims to guide complete beginners to web development with all that they need to start coding websites.

The aim of this area of MDN is not to take you from "beginner" to "expert" but to take you from "beginner" to "comfortable." From there, you should be able to start making your way, learning from the rest of MDN, and other intermediate to advanced resources that assume a lot of previous knowledge.

If you are a complete beginner, web development can be challenging — we will hold your hand and provide enough detail for you to feel comfortable and learn the topics properly. You should feel at home whether you are a student learning web development (on your own or as part of a class), a teacher looking for class materials, a hobbyist, or someone who just wants to understand more about how web technologies work.

Looking to become a front-end web developer?

We have put together a course that includes a
towards your goal.

Get started

In this article

- Where to start
- Topics covered
- Getting our code examples
- Contact us
- See also



How to uplevel your hybrid cloud connection on Equinix Metal using Equinix Fabric

Mozilla ads

https://tinyurl.com/mryx9yzw

A first splash into JavaScript

Lecture: A first splash int JavaScript
Module: JavaScript first steps

Filter

JavaScript — Dynamic client-side scripting

- JavaScript first steps
  - JavaScript First Steps
  - What is JavaScript?
  - A first splash into JavaScript
  - What went wrong? Troubleshooting JavaScript
  - Storing the information you need — Variables
  - Basic math in JavaScript — numbers and operators
  - Handling text — strings in JavaScript
  - Useful string methods
  - Arrays
  - Silly story generator
- JavaScript building blocks
- Introducing JavaScript objects
- Asynchronous JavaScript
- Client-side web APIs

A first splash into JavaScript

Previous Overview: First steps Next

Now you've learned something about the theory of JavaScript and what you can do with it, we are going to give you an idea of what the process of creating a simple JavaScript program is like, by guiding you through a practical tutorial. Here you'll build up a simple "Guess the number" game, step by step.

Prerequisites:	Basic computer literacy, a basic understanding of HTML and CSS, an understanding of what JavaScript is.
Objective:	To have a first bit of experience at writing some JavaScript, and gain at least a basic understanding of what writing a JavaScript program involves.

We want to set really clear expectations here: You won't be expected to learn JavaScript by the end of this article, or even understand all the code we are asking you to write. Instead, we want to give you an idea of how JavaScript's features work together, and what writing JavaScript feels like. In subsequent articles you'll revisit all the features shown here in a lot more detail, so don't worry if you don't understand it all immediately!

Note: Many of the code features you'll see in programming languages — functions, loops, etc. The code syntax looks different, but the concepts are still largely the same.

In this article

- Thinking like a programmer
- Example — Guess the number game
- Finished for now...

Check out the new Complete Intro to React course and learn to build real-world apps.

Mozilla ads

Don't want to see ads?

https://tinyurl.com/mvrpftr3

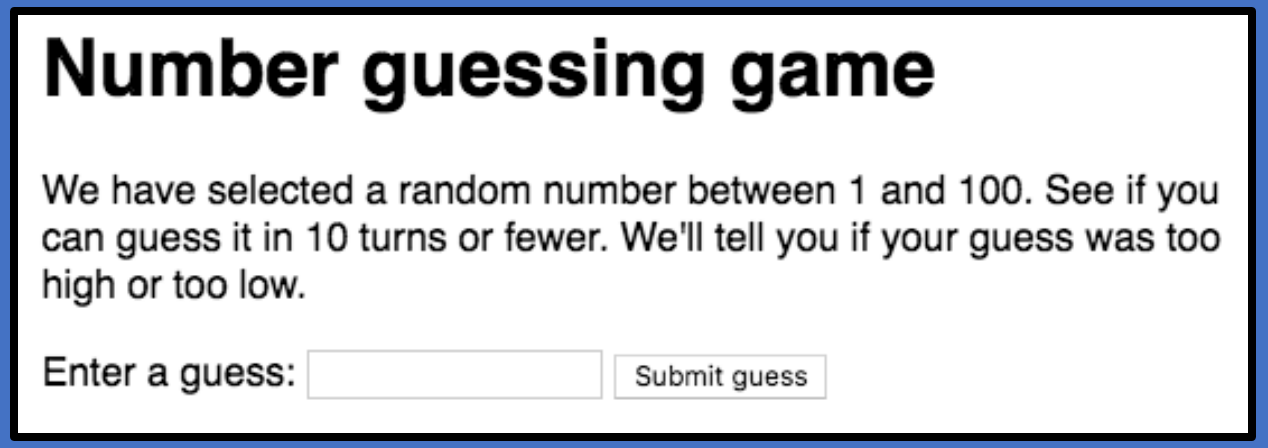
Objective of the lecture

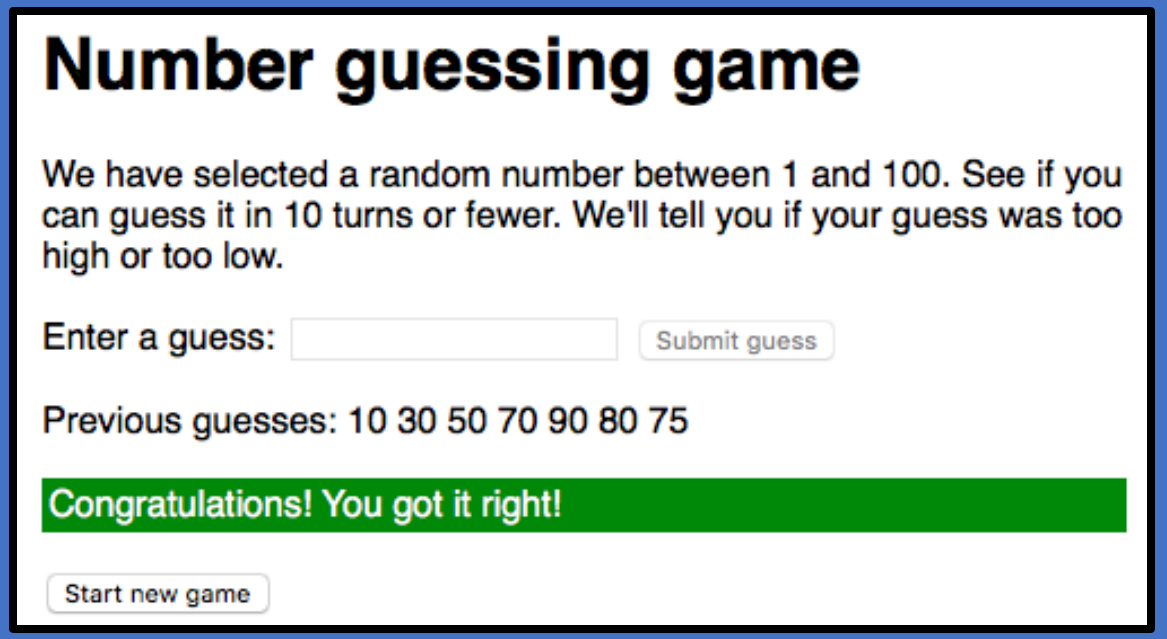
Objective: To have a **first bit** of **experience** at writing some **JavaScript** and **gain** at least a basic **understanding** of what **writing** a **JavaScript program** involves.

Introduction

Introduction

- We are going to give you a **crash course** in the **basic features** of **JavaScript** via a **practical tutorial**.
- You'll **build** up a simple **"Guess the number" game**, **step by step**.





Note

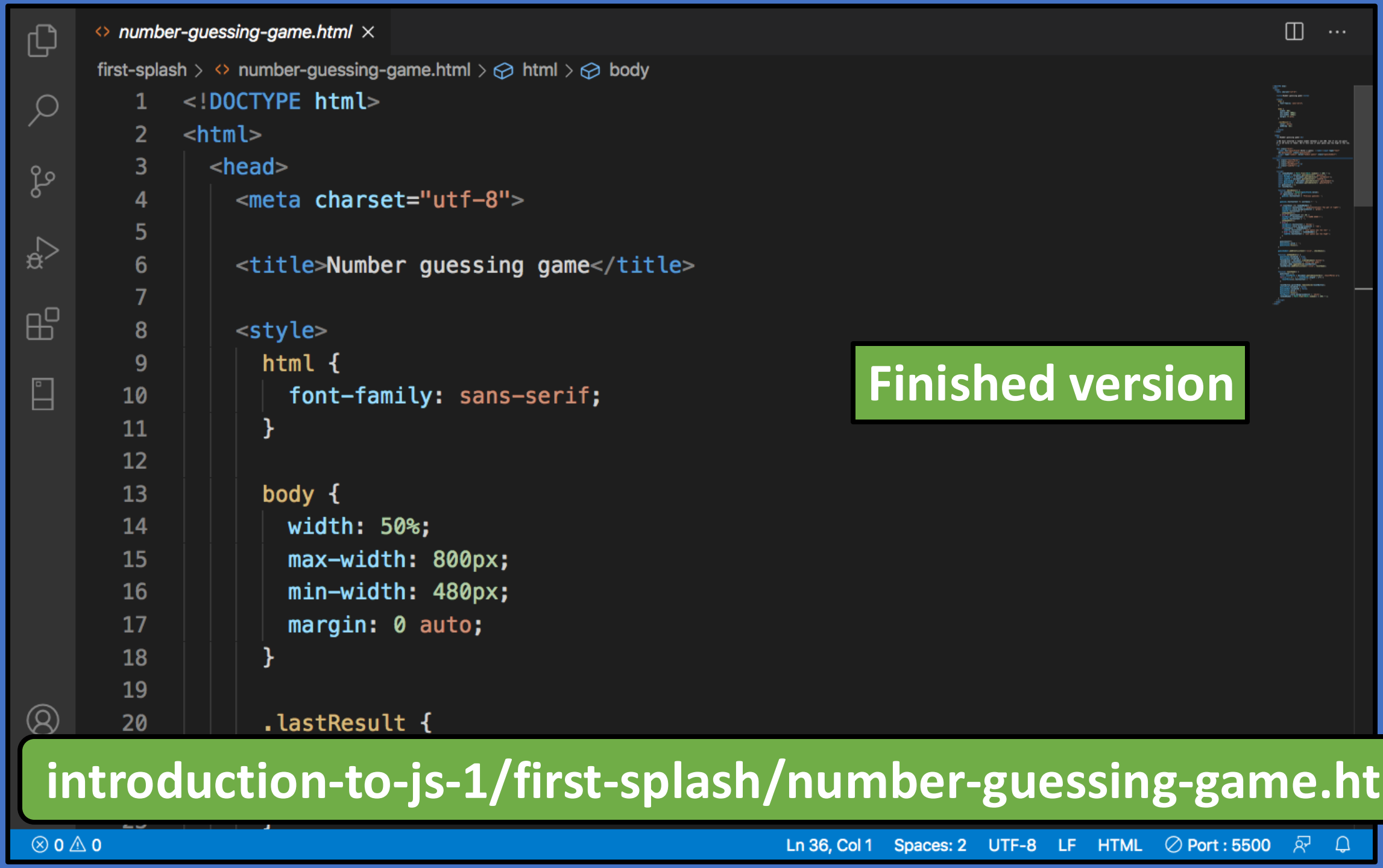
Note: Many of the **code features** you'll see in **JavaScript** are the **same** as in other **programming languages** — **functions**, **loops**, etc. The **code syntax looks different**, but the **concepts** are still **largely** the **same**.

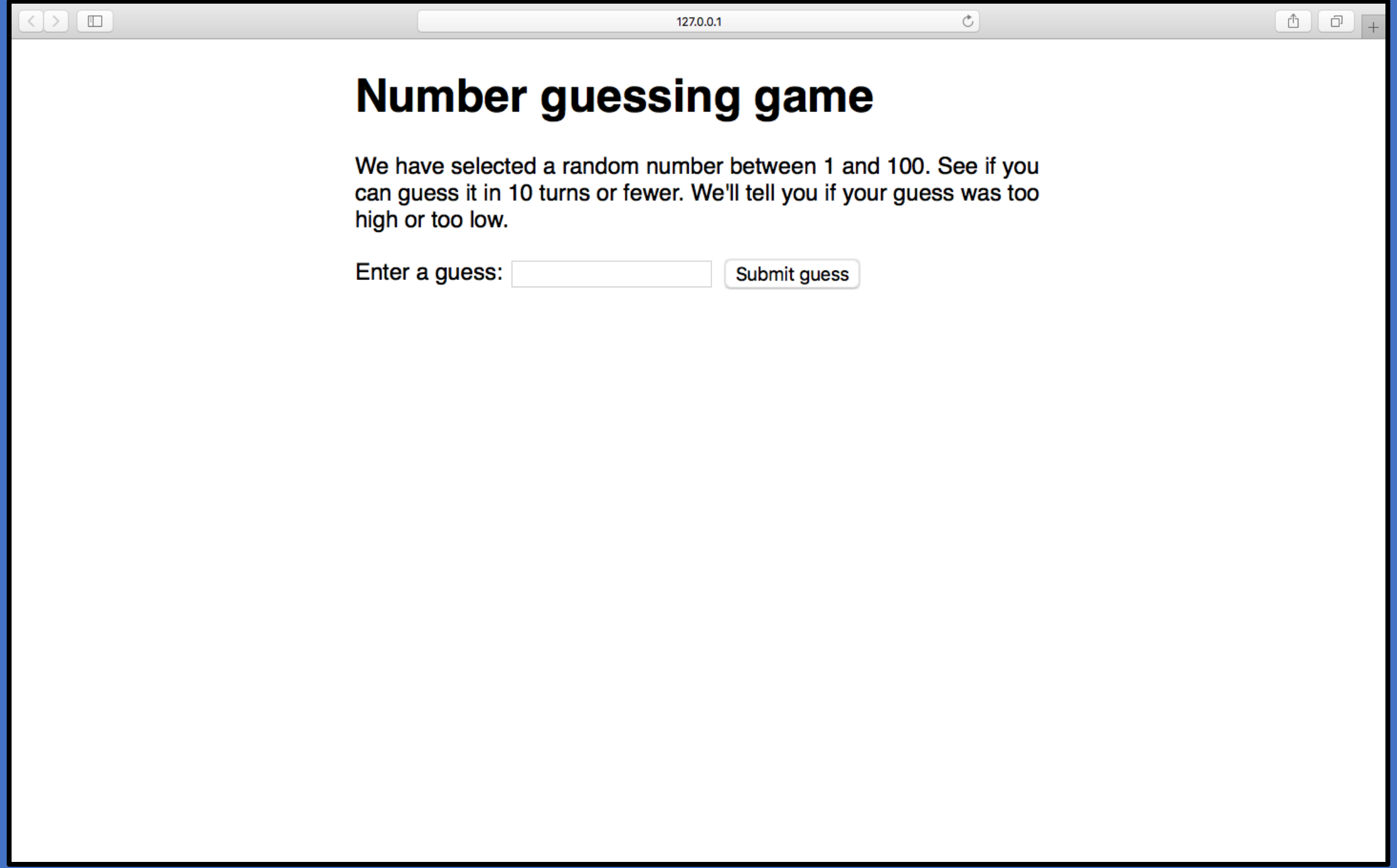
Thinking like a programmer

Thinking like a programmer

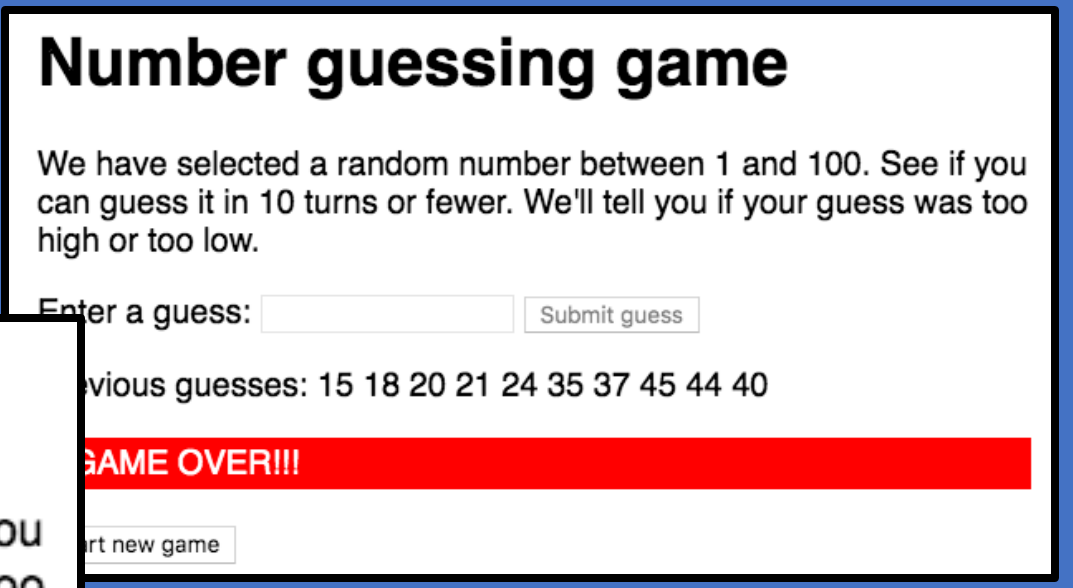
- **Involves**:
  - ✓ **looking** at **descriptions** of **WHAT** your **program needs** to **do**,
  - ✓ **working out** what **code features** are **needed** to **achieve** those **things**, and how to **make them work together**.
- **Requires**:
  - ✓ A **mixture** of **hard work**, **experience** with the **programming syntax**, and **practice**.
    - Plus a **bit** of **creativity**.
- The **more you code**, the **better you'll get** at it.

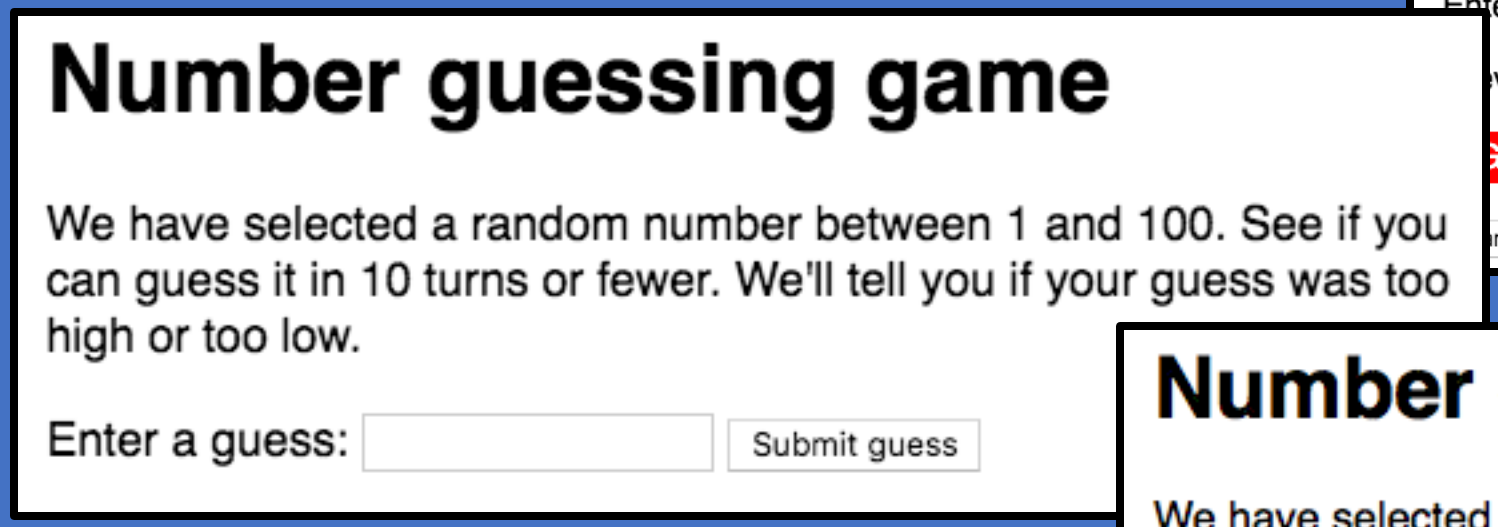
Example:
Guess the number game

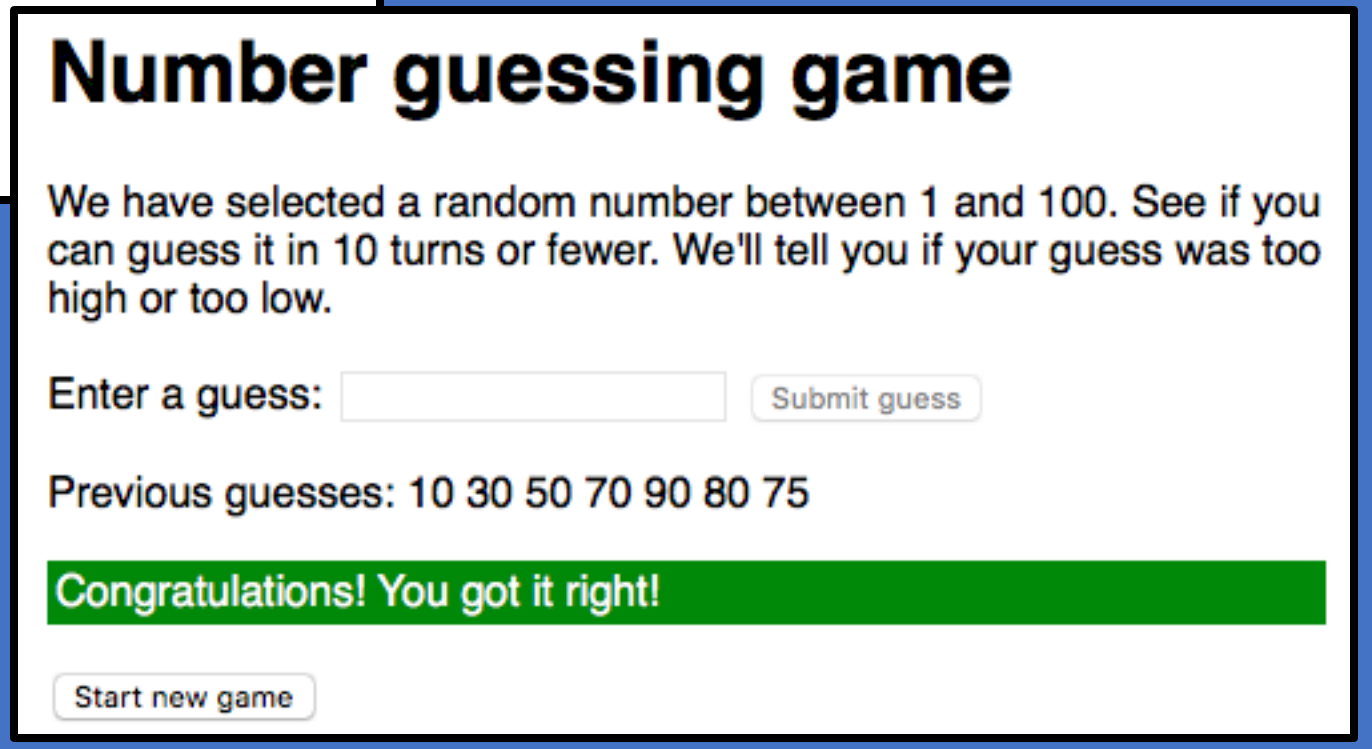

number-guessing-game.html
first-splash > number-guessing-game.html > html > body
1 <!DOCTYPE html>
2 <html>
3 <head>
4 <meta charset="utf-8">
5
6 <title>Number guessing game</title>
7
8 <style>
9 html {
10 font-family: sans-serif;
11 }
12
13 body {
14 width: 50%;
15 max-width: 800px;
16 min-width: 480px;
17 margin: 0 auto;
18 }
19
20 .lastResult {
Finished version
introduction-to-js-1/first-splash/number-guessing-game.html
0 0
Ln 36, Col 1
Spaces: 2
UTF-8
LF
HTML
Port : 5500


127.0.0.1
Number guessing game
We have selected a random number between 1 and 100. See if you can guess it in 10 turns or fewer. We'll tell you if your guess was too high or too low.
Enter a guess:
Submit guess

Guess the number game







https://mzl.la/3aS8RMs

Functional requirements & algorithm

Functional requirements

- I want you to **create** a simple "**guess the number**" **type game**.
- It should **choose** a **random number between 1 and 100**, then **challenge** the **player** to **guess** the **number** in **10** turns.
- **After each turn** the **player** should be **told** if he/she is **right** or **wrong**
  - ✓ If **wrong**, **indicate whether guess** was **too low** or **too high**.

Functional requirements (cont.)

- It should also **tell** the **player what numbers** he/she **previously guessed**.
- The **game** will **end once** the **player guesses correctly**, **or** once they **run out of turns**.
- When the **game ends**, the **player** should be given an **option** to **start playing again**.

The algorithm in 9 steps

1. **Generate** a **random number** between **1** and **100**.
2. **Record** the **turn number** the **player is** on.
   - ✓ **Start** it on **1**.
3. Provide the **player** with a **way** to **guess** what the **number is**.
4. **Once** a **guess** has been **submitted** first **record it** somewhere
   - ✓So the **user** can **see** their **previous guesses**.
5. **Check whether** it is the **correct number**.

The algorithm in 9 steps

6. If it is **correct**:
   - ✓**Display congratulations** message.
   - ✓**Stop** the **player** from being able to **enter more guesses**
     - This would **mess the game up**.
   - ✓**Display control** allowing the **player** to **restart** the **game**.

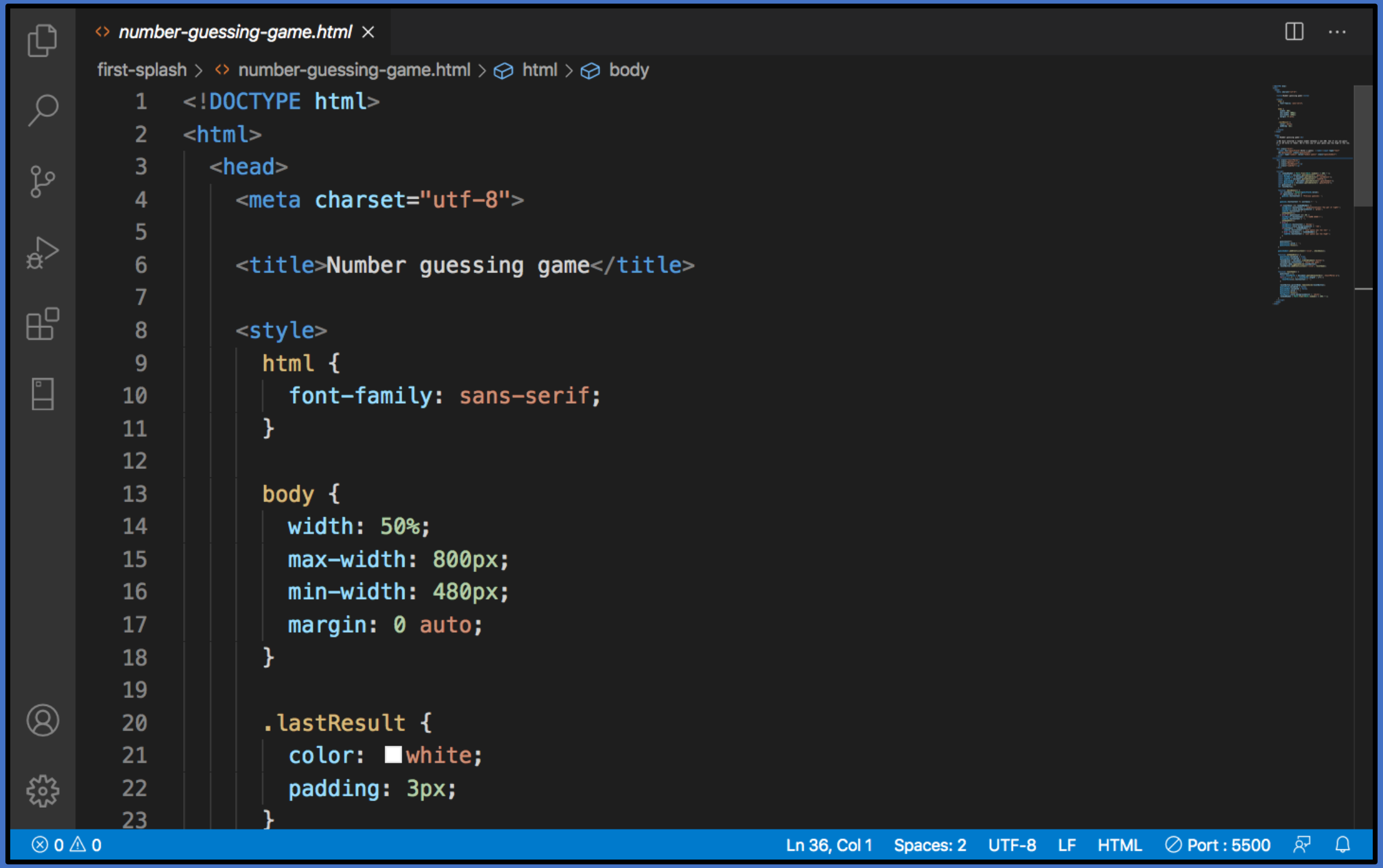
7. If it is **wrong AND** the player has **turns left**:
   - ✓Tell the **player** they are **wrong**.
   - ✓**Allow** them to **enter** another **guess**.
   - ✓**Increment** the **turn number** by **1**.

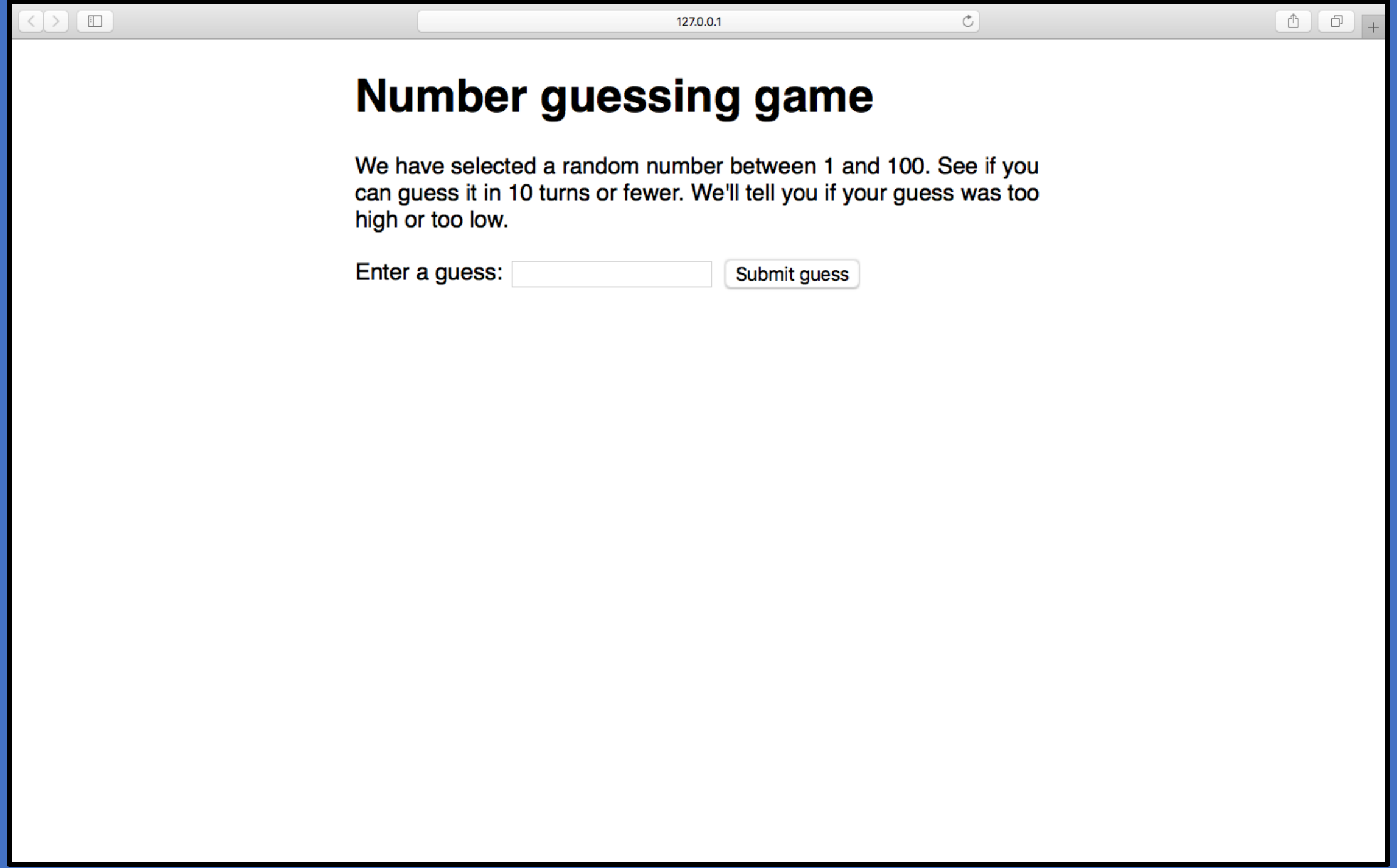
The algorithm in 9 steps

8. If it is **wrong AND** the **player** has **no turns left**:
   - ✓Tell the **player** it is **game over**.
   - ✓**Stop** the **player** from being able to **enter** more **guesses**.
     - This would **mess the game up**.
   - ✓**Display control allowing** the **player** to **restart the game**.

9. **Once** the **game restarts**, make **sure** the **game logic** and **UI** are completely **reset**, then go **back to step 1**.

The source-code of the finished version


number-guessing-game.html
first-splash > number-guessing-game.html > html > body
1 <!DOCTYPE html>
2 <html>
3 <head>
4 <meta charset="utf-8">
5
6 <title>Number guessing game</title>
7
8 <style>
9 html {
10 font-family: sans-serif;
11 }
12
13 body {
14 width: 50%;
15 max-width: 800px;
16 min-width: 480px;
17 margin: 0 auto;
18 }
19
20 .lastResult {
21 color: white;
22 padding: 3px;
23 }
0 0
Ln 36, Col 1
Spaces: 2
UTF-8
LF
HTML
Port : 5500


127.0.0.1
Number guessing game
We have selected a random number between 1 and 100. See if you can guess it in 10 turns or fewer. We'll tell you if your guess was too high or too low.
Enter a guess:
Submit guess

number-guessing-game.html

first-splash > number-guessing-game.html > html > body

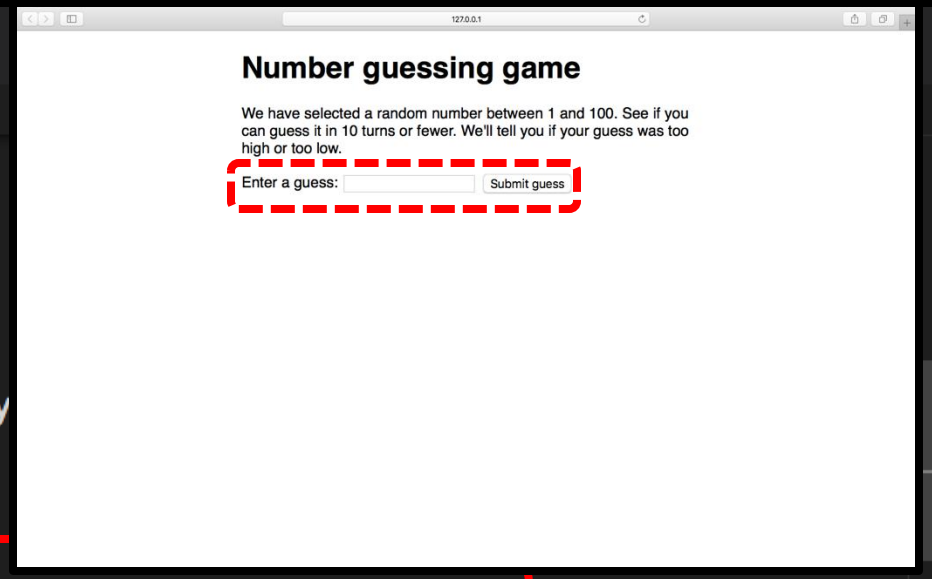
```
3   <head>
4     <meta charset="utf-8">
5
6     <title>Number guessing game</title>
7
8     <style>
9       html {
10        font-family: sans-serif;
11      }
12
13      body {
14        width: 50%;
15        max-width: 800px;
16        min-width: 480px;
17        margin: 0 auto;
18      }
19
20      .lastResult {
21        color: white;
22        padding: 3px;
23      }
24    </style>
25  </head>
```

We **use internal CSS style** in this **example**

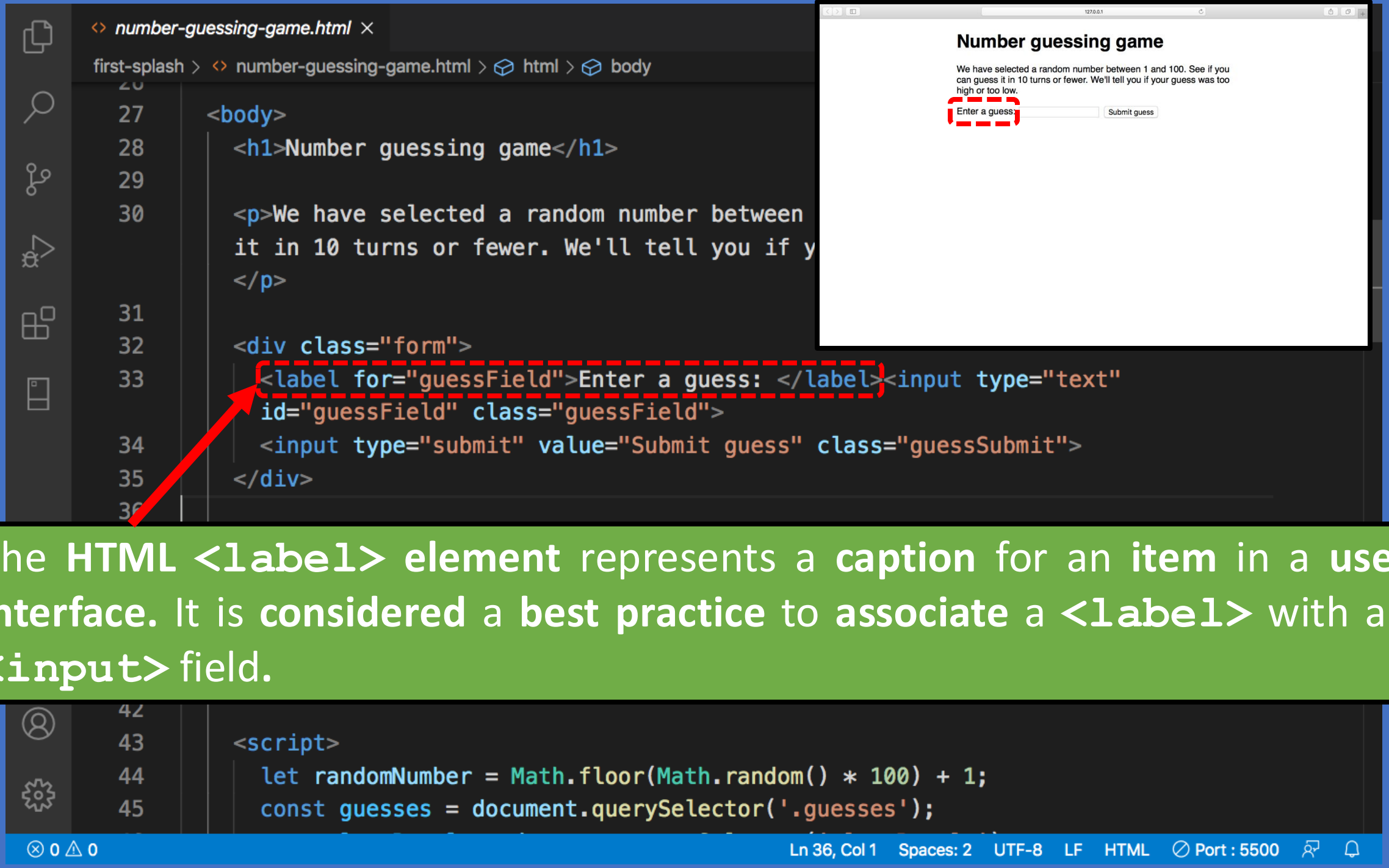
number-guessing-game.html

first-splash > number-guessing-game.html > html > body

```
27  <body>
28    <h1>Number guessing game</h1>
29
30    <p>We have selected a random number between
      it in 10 turns or fewer. We'll tell you if y
      </p>
31
32    <div class="form">
33      <label for="guessField">Enter a guess: </label><input type="text"
        id="guessField" class="guessField">
34      <input type="submit" value="Submit guess" class="guessSubmit">
35    </div>
36
41    </div>
42
43    <script>
44      let randomNumber = Math.floor(Math.random() * 100) + 1;
45      const guesses = document.querySelector('.guesses');
```



Our **example** has a **kind** of **form** that **allows** the **user** to **enter** and **submit** a **guess**.


number-guessing-game.html
first-splash > number-guessing-game.html > html > body
27 <body>
28 <h1>Number guessing game</h1>
29
30 We have selected a random number between
it in 10 turns or fewer. We'll tell you if y

31
32
33 <label for="guessField">Enter a guess: </label><input type="text"
id="guessField" class="guessField">
34 <input type="submit" value="Submit guess" class="guessSubmit">
35
43 <script>
44 let randomNumber = Math.floor(Math.random() * 100) + 1;
45 const guesses = document.querySelector('.guesses');
Number guessing game
We have selected a random number between 1 and 100. See if you can guess it in 10 turns or fewer. We'll tell you if your guess was too high or too low.
Enter a guess:
Submit guess
The HTML <label> element represents a caption for an item in a user interface. It is considered a best practice to associate a <label> with an <input> field.
Ln 36, Col 1
Spaces: 2
UTF-8
LF
HTML
Port : 5500

<label>

On this Page

- Attributes
- Styling with CSS
- Examples
- Accessibility concerns
- Technical summary
- Specifications
- Browser compatibility

The **HTML `<label>` element** represents a caption for an item in a user interface.

HTML Demo: <label>

Reset

HTML | CSS

```
1  <div class="preference">
2      <label for="cheese">Do you like cheese?</label>
3      <input type="checkbox" name="cheese" id="cheese">
4  </div>
5
6  <div class="preference">
7      <label for="peas">Do you like peas?</label>
8      <input type="checkbox" name="peas" id="peas">
9  </div>
10
```

Output

Do you like cheese? ☐

Do you like peas? ☐

Associating a `<label>` with an `<input>` element offers some major advantages:

- The label text is not only visually associated with its corresp
  programmatically associated with it too. This means that, for
  read out the label when the user is focused on the form input,
  assistive technology user to understand what data should be entered.

Related Topics

<button>

<datalist>

<fieldset>

<form>

<input>

<label>

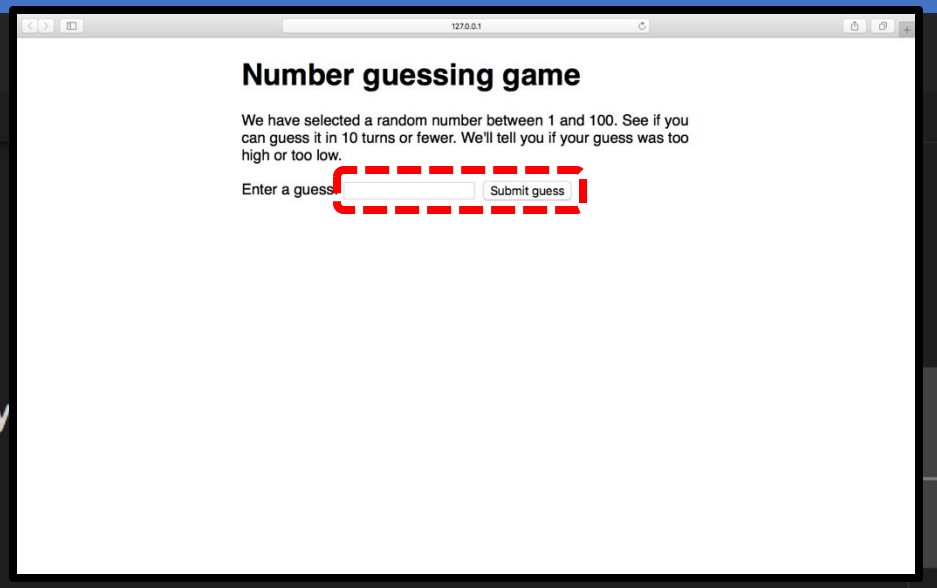
<legend>

https://mzl.la/331KJp2

number-guessing-game.html

first-splash > number-guessing-game.html > html > body

```
27  <body>
28    <h1>Number guessing game</h1>
29
30    <p>We have selected a random number between
      it in 10 turns or fewer. We'll tell you if y
      </p>
31
32    <div class="form">
33      <label for="guessField">Enter a guess: </label><input type="text"
        id="guessField" class="guessField">
34      <input type="submit" value="Submit guess" class="guessSubmit">
35    </div>
36
41  </div>
42
43  <script>
44    let randomNumber = Math.floor(Math.random() * 100) + 1;
45    const guesses = document.querySelector('.guesses');
```



The **HTML** `<input>` **element** is used to **create interactive controls** for **web-based forms** in **order** to **accept data** from the **user.**

<input>: The Input (Form Input) element

On this Page

- <input> types
- Attributes
- Methods
- CSS
- Additional features
- Specifications
- Accessibility concerns
- Browser compatibility
- See also

The **HTML `<input>` element** is used to create interactive controls for web-based forms in order to accept data from the user; a wide variety of types of input data and control widgets are available, depending on the device and user agent. The `<input>` element is one of the most powerful and complex in all of HTML due to the sheer number of combinations of input types and attributes.

HTML Demo: <input type="text">

Reset

HTML | CSS

```
1 <label for="name">Name (4 to 8 characters):</label>
2
3 <input type="text" id="name" name="name" required
4        minlength="4" maxlength="8" size="10">
5
```

Output

Name (4 to 8 characters):

Related Topics

<button>

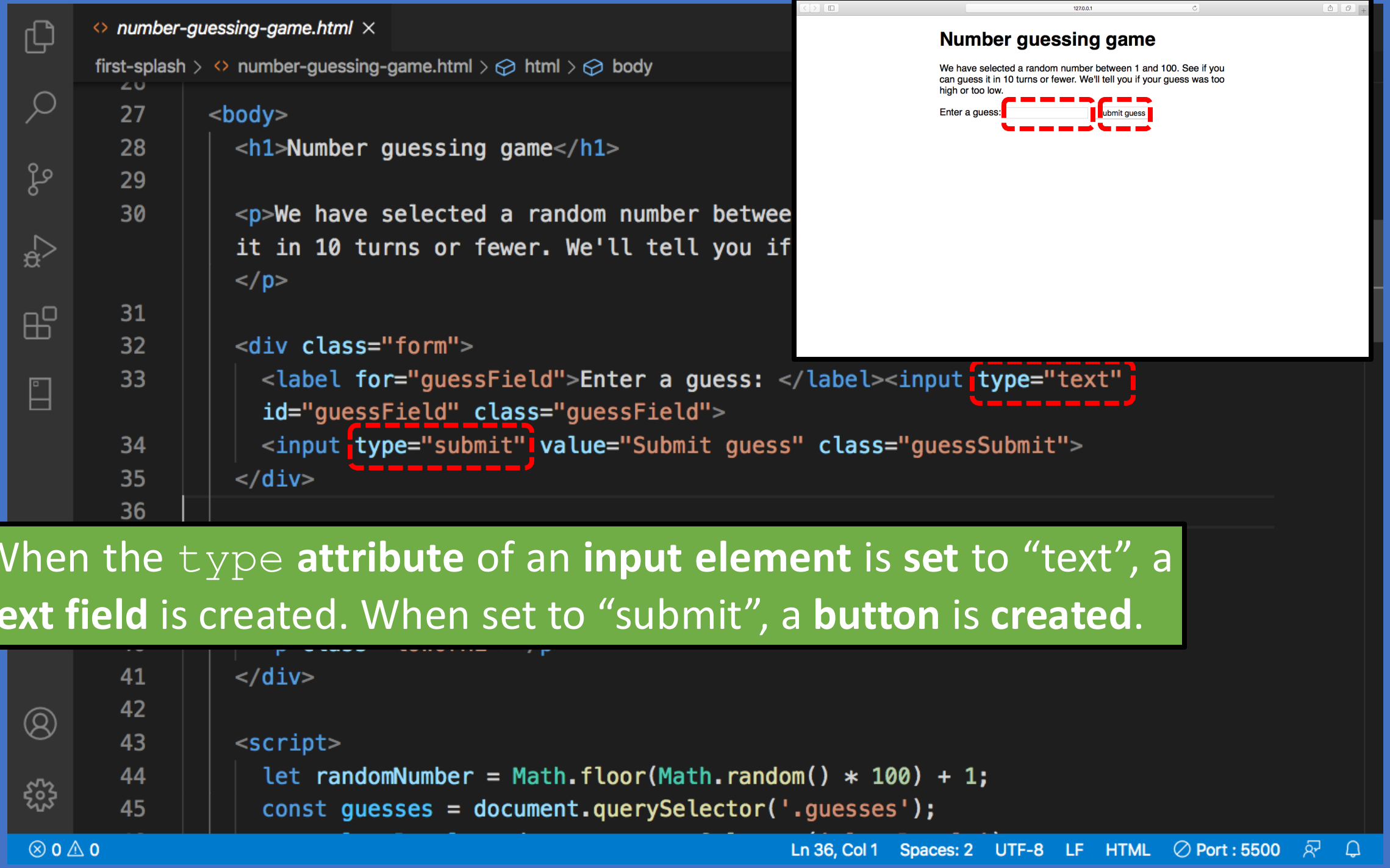
<datalist>

<fieldset>

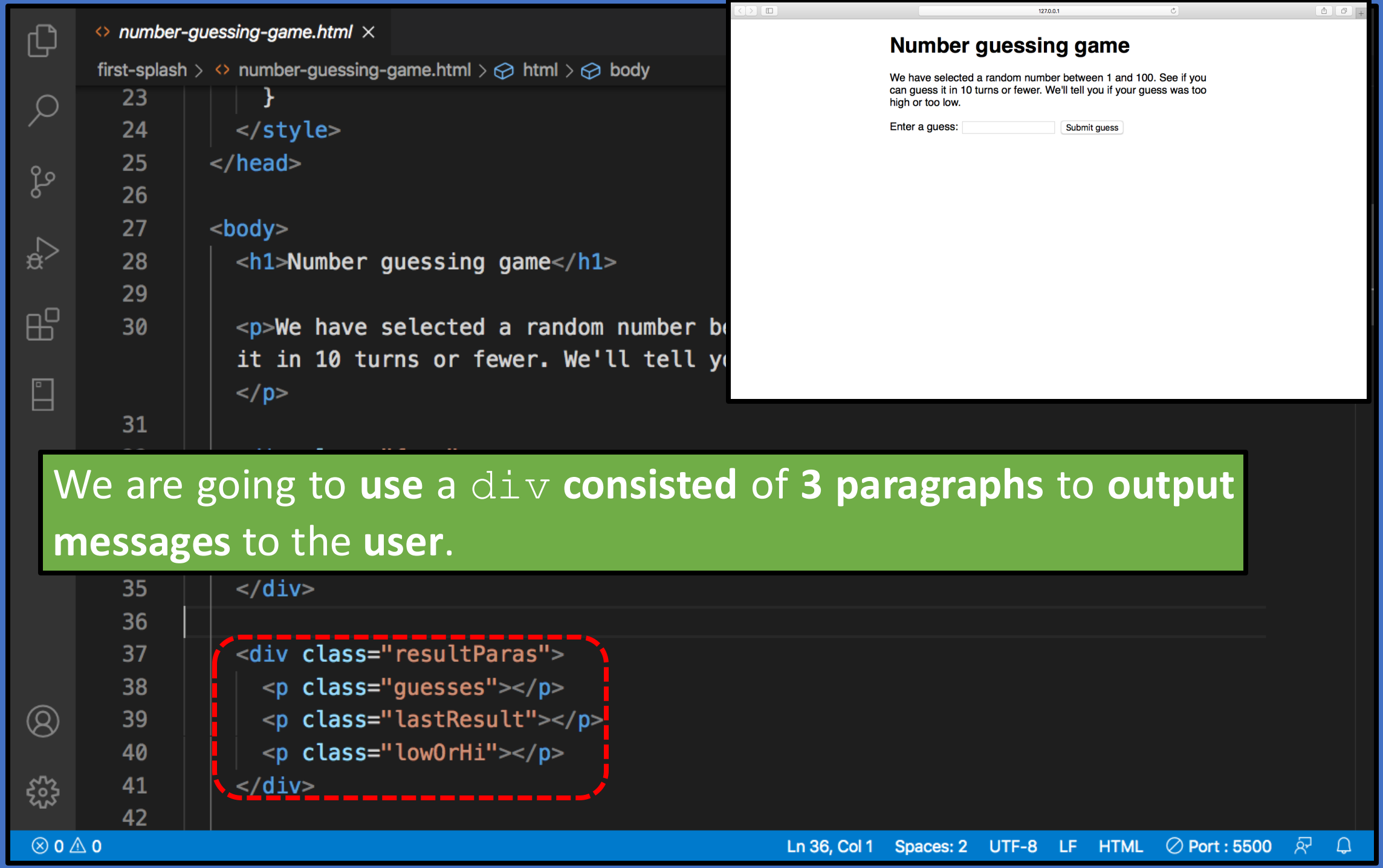
<form>

<input>

https://mzl.la/35cRabp

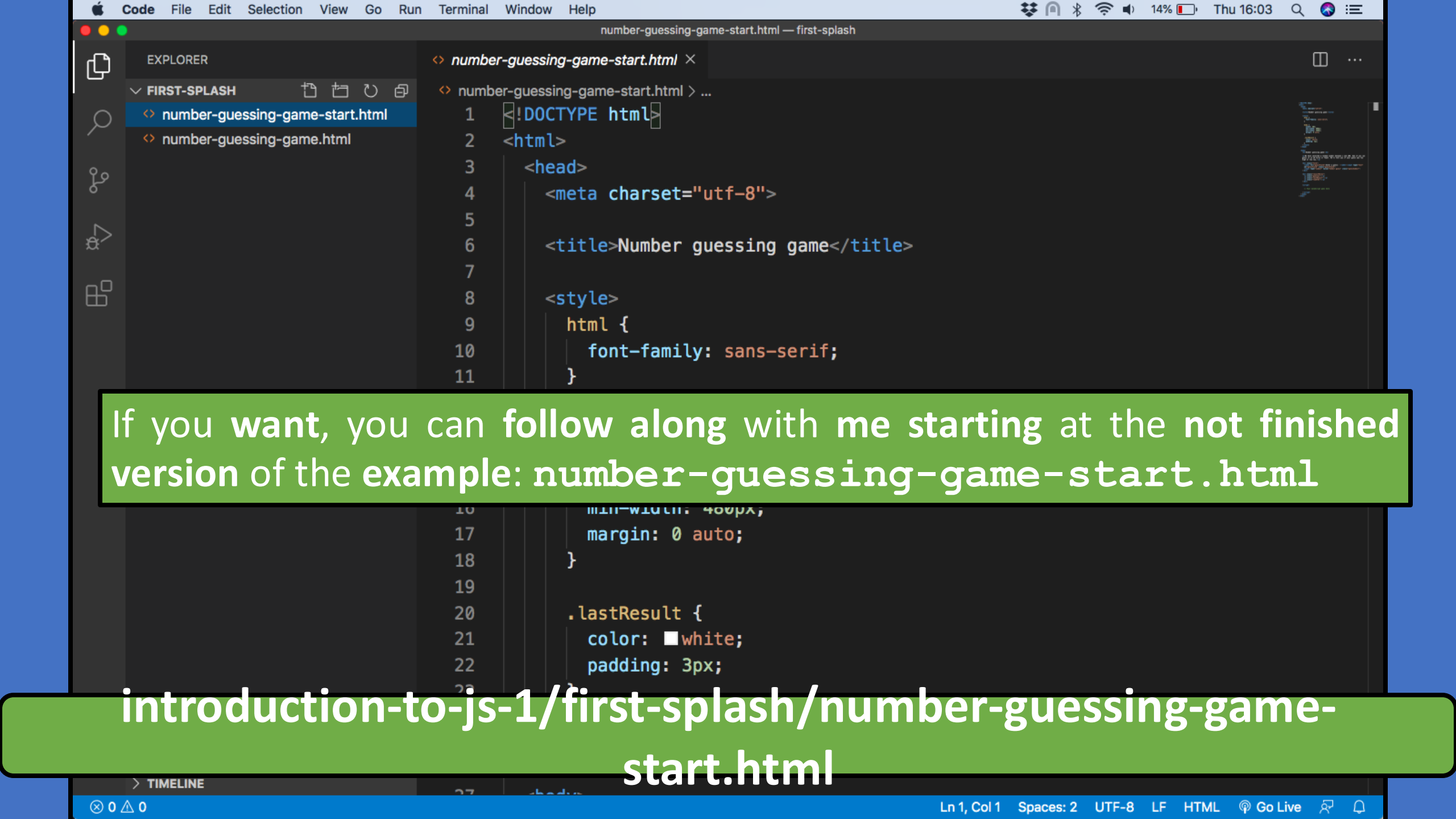

number-guessing-game.html
first-splash > number-guessing-game.html > html > body
27 <body>
28 <h1>Number guessing game</h1>
29
30 We have selected a random number betwee
it in 10 turns or fewer. We'll tell you if

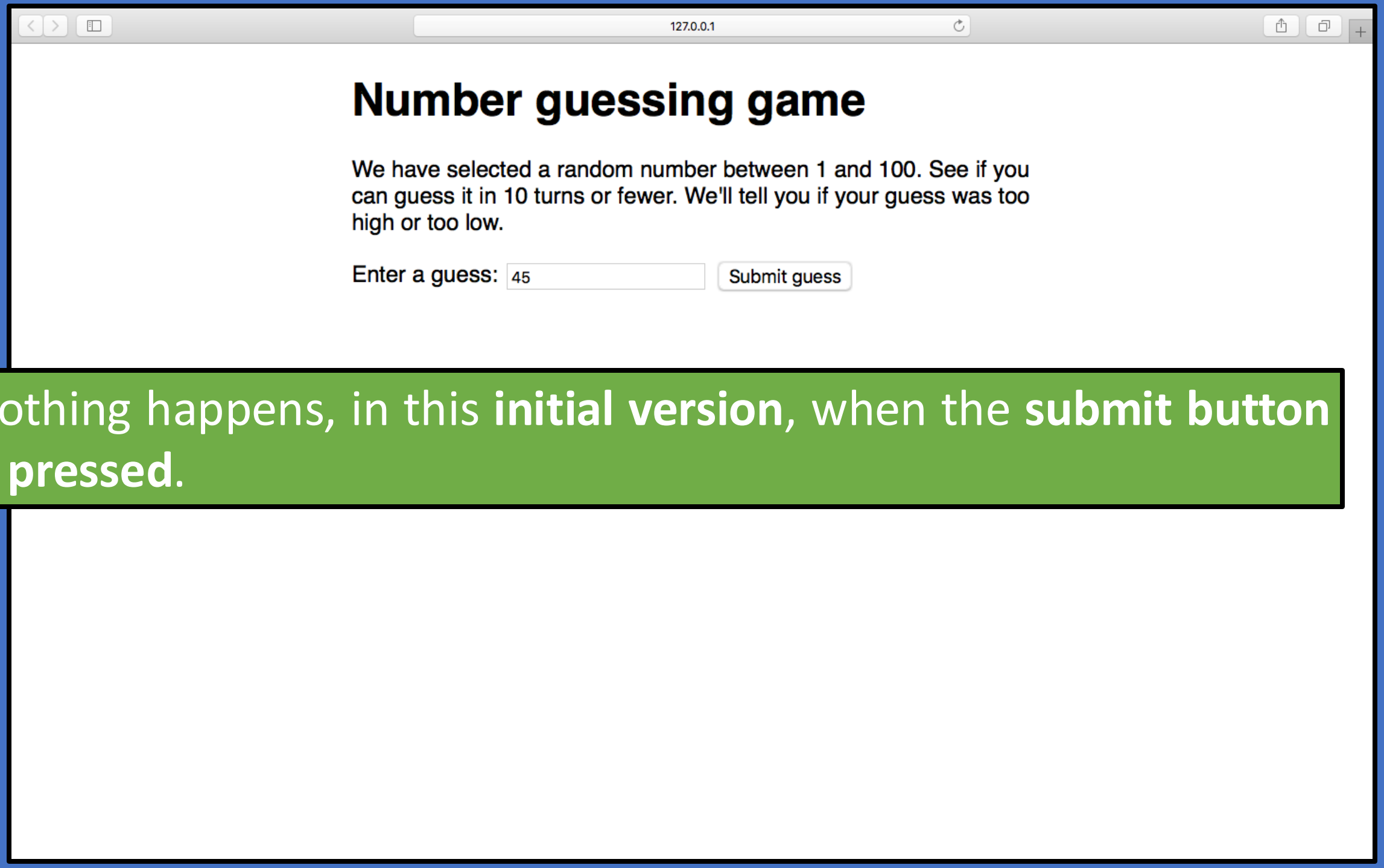
31
32
33 <label for="guessField">Enter a guess: </label><input type="text"
id="guessField" class="guessField">
34 <input type="submit" value="Submit guess" class="guessSubmit">
35
36
41
42
43 <script>
44 let randomNumber = Math.floor(Math.random() * 100) + 1;
45 const guesses = document.querySelector('.guesses');
Number guessing game
We have selected a random number between 1 and 100. See if you can guess it in 10 turns or fewer. We'll tell you if your guess was too high or too low.
Enter a guess:
Submit guess
When the type attribute of an input element is set to "text", a text field is created. When set to "submit", a button is created.
0 0
Ln 36, Col 1
Spaces: 2
UTF-8
LF
HTML
Port : 5500


number-guessing-game.html
first-splash > number-guessing-game.html > html > body
23 }
24 </style>
25 </head>
26
27 <body>
28 <h1>Number guessing game</h1>
29
30 We have selected a random number b
it in 10 turns or fewer. We'll tell y

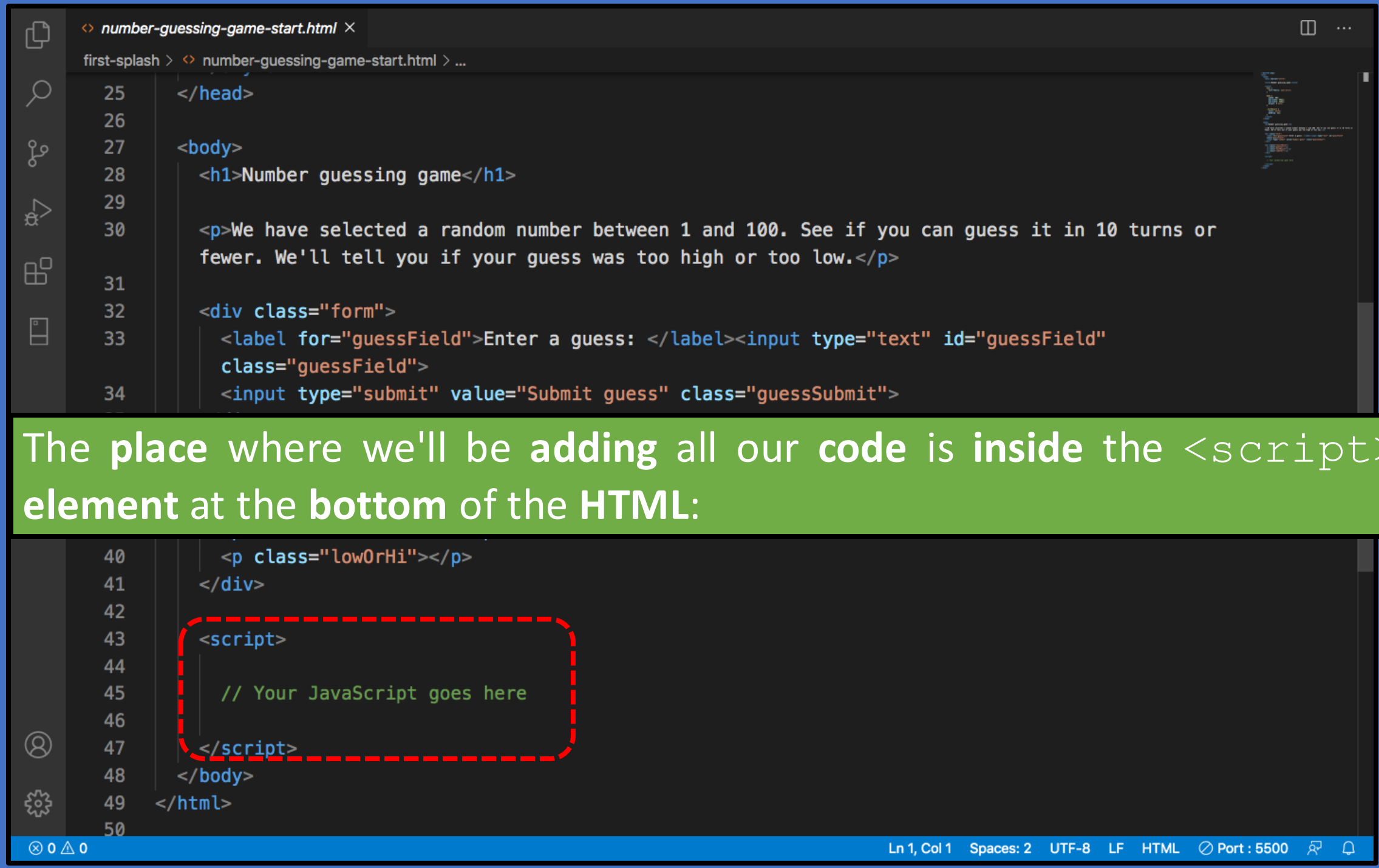
31
127.0.0.1
Number guessing game
We have selected a random number between 1 and 100. See if you can guess it in 10 turns or fewer. We'll tell you if your guess was too high or too low.
Enter a guess:
Submit guess
We are going to use a div consisted of 3 paragraphs to output messages to the user.
35
36
37
38
39
40
41
42
0 0
Ln 36, Col 1
Spaces: 2
UTF-8
LF
HTML
Port : 5500

The source-code of the initial version


Code File Edit Selection View Go Run Terminal Window Help
14% Thu 16:03
number-guessing-game-start.html — first-splash
EXPLORER
FIRST-SPLASH
number-guessing-game-start.html
number-guessing-game.html
number-guessing-game-start.html
number-guessing-game-start.html > ...
1 <!DOCTYPE html>
2 <html>
3 <head>
4 <meta charset="utf-8">
5
6 <title>Number guessing game</title>
7
8 <style>
9 html {
10 font-family: sans-serif;
11 }
If you want, you can follow along with me starting at the not finished version of the example: number-guessing-game-start.html
17 margin: 0 auto;
18 }
19
20 .lastResult {
21 color: white;
22 padding: 3px;
TIMELINE
0 0
Ln 1, Col 1 Spaces: 2 UTF-8 LF HTML Go Live
introduction-to-js-1/first-splash/number-guessing-game-start.html



Nothing happens, in this **initial version**, when the **submit button** is **pressed**.


number-guessing-game-start.html
first-splash > number-guessing-game-start.html > ...
25 </head>
26
27 <body>
28 <h1>Number guessing game</h1>
29
30 We have selected a random number between 1 and 100. See if you can guess it in 10 turns or fewer. We'll tell you if your guess was too high or too low.
31
32
33 <label for="guessField">Enter a guess: </label><input type="text" id="guessField" class="guessField">
34 <input type="submit" value="Submit guess" class="guessSubmit">
The place where we'll be adding all our code is inside the <script> element at the bottom of the HTML:
40
41
42
43 <script>
44
45 // Your JavaScript goes here
46
47 </script>
48 </body>
49 </html>
50
0 0
Ln 1, Col 1 Spaces: 2 UTF-8 LF HTML Port : 5500

Variables

Adding variables to store our data

- **Add** the following **lines** inside your `<script>` **element.**

```
01  let randomNumber = Math.floor(Math.random() * 100) + 1;
02
03  const guesses = document.querySelector('.guesses’);
04  const lastResult = document.querySelector('.lastResult’);
05  const lowOrHi = document.querySelector('.lowOrHi’);
06
07  const guessSubmit = document.querySelector('.guessSubmit');
08  const guessField = document.querySelector('.guessField’);
09
10  let guessCount = 1;
11  let resetButton;
```

JavaScript

HTML

```
1  <div class="resultParas">
2      <p class="guesses"></p>
3      <p class="lastResult"></p>
4      <p class="lowOrHi"></p>
5  </div>
```

HTML

```
1  <label for="guessField">Enter a guess: </label>
2  <input type="text" id="guessField" class="guessField">
3  <input type="submit" value="Submit guess" class="guessSubmit">
```

Adding variables to store our data

The **`Math.random()` function** returns a **floating-point, pseudo-random number** in the range **0** to less than **1** (inclusive of 0, but not 1) with approximately **uniform distribution** over that range.

```
01 let randomNumber = Math.floor(Math.random() * 100) + 1;
02
03 const guesses = document.querySelector('.guesses’);
04 const lastResult = document.querySelector('.lastResult’);
05 const lowOrHi = document.querySelector('.lowOrHi’);
06
07 const guessSubmit = document.querySelector('.guessSubmit');
08 const guessField = document.querySelector('.guessField’);
09
10 let guessCount = 1;
11 let resetButton;
```

JavaScript

HTML

```
1 <div class="resultParas">
2     <p class="guesses"></p>
3     <p class="lastResult"></p>
4     <p class="lowOrHi"></p>
5 </div>
```

HTML

```
1 <label for="guessField">Enter a guess: </label>
2 <input type="text" id="guessField" class="guessField">
3 <input type="submit" value="Submit guess" class="guessSubmit">
```

Adding variables to store our data

- **Add** the following **lines** inside your `<script>` **element.**

```
01  let randomNumber = Math.floor(Math.random() * 100) + 1;
02
03  const guesses = document.querySelector('.guesses');
04  const lastResult = document.querySelector('.lastResult');
05  const lowOrHi = document.querySelector('.lowOrHi');
06
07  const guessSubmit = document.querySelector('.guessSubmit');
08  const guessField = document.querySelector('.guessField');
09
10  let guessCount = 1;
11  let resetButton;
```

JavaScript

HTML

```
1  <div class="resultParas">
2      <p class="guesses"></p>
3      <p class="lastResult"></p>
4      <p class="lowOrHi"></p>
5  </div>
```

HTML

```
1  <label for="guessField">Enter a guess: </label>
2  <input type="text" id="guessField" class="guessField">
3  <input type="submit" value="Submit guess" class="guessSubmit">
```

Adding variables to store our data

- The **first three constants** are each **made** to **store** a **reference** to the **results paragraphs** in our **HTML**.
  - ✓They are **used** to **insert values** into the **paragraphs** later on in the **code**.

- Note how **they are inside a** `<div>` **element**, which is itself **used** to **select all three later on** for **resetting**, when we **restart** the **game**.

```
01  let randomNumber = Math.floor(Math.random() * 100) + 1;
02
03  const guesses = document.querySelector('.guesses');
04  const lastResult = document.querySelector('.lastResult');
05  const lowOrHi = document.querySelector('.lowOrHi');
06
07  const guessSubmit = document.querySelector('.guessSubmit');
08  const guessField = document.querySelector('.guessField');
```

```
1  <div class="resultParas">
2      <p class="guesses"></p>
3      <p class="lastResult"></p>
4      <p class="lowOrHi"></p>
5  </div>
```

Adding variables to store our data

- The **next two constants** store **references** to the **form text input** and **submit button** and are **used** to **control submitting the guess** later on.

```
01  let randomNumber = Math.floor(Math.random() * 100) + 1;
02
03  const guesses = document.querySelector('.guesses’);
04  const lastResult = document.querySelector('.lastResult’);
05  const lowOrHi = document.querySelector('.lowOrHi’);
06
07  const guessSubmit = document.querySelector('.guessSubmit');
08  const guessField = document.querySelector('.guessField’);
09
10  let guessCount = 1;
11  let resetButton;
```

JavaScript

```
1  <label for="guessField">Enter a guess: </label>
2  <input type="text" id="guessField" class="guessField">
3  <input type="submit" value="Submit guess" class="guessSubmit">
```

HTML

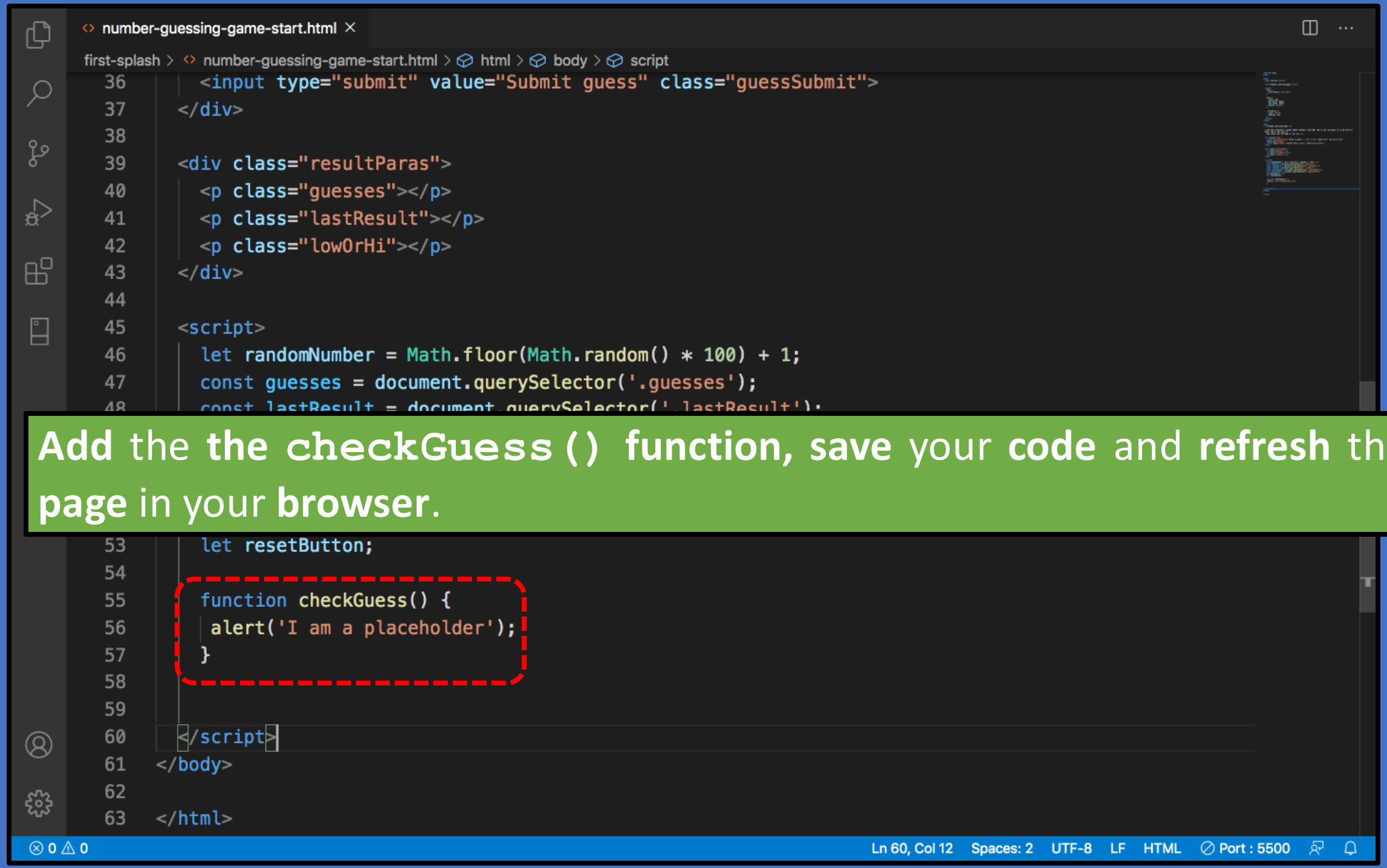
Adding variables to store our data

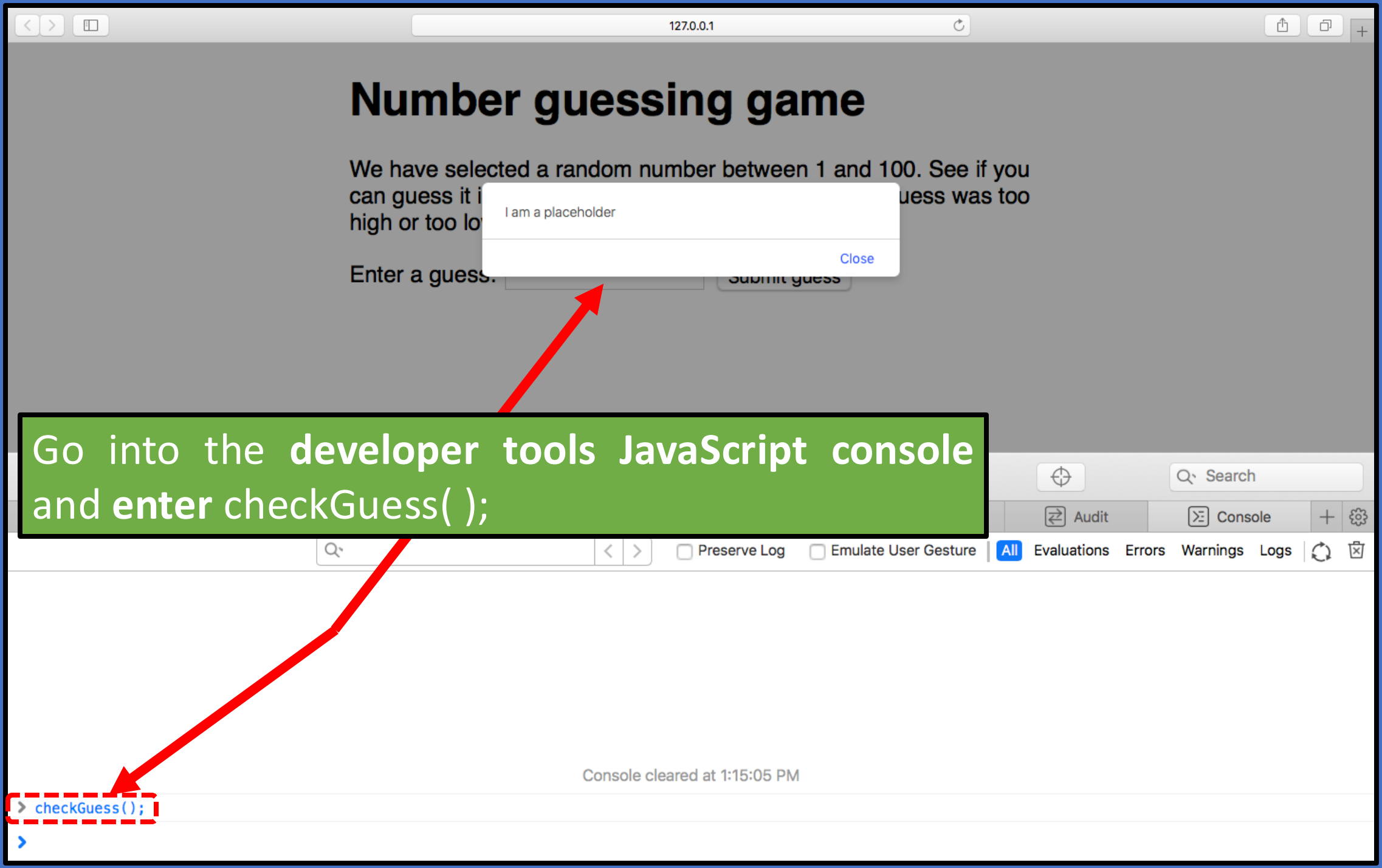
- Our **final two variables** store a **guess count** of **1** (used to **keep track** of **how many guesses** the **player** has **had**), and a **reference** to a **reset button** that **doesn't exist yet (**but will **later**).

```
01  let randomNumber = Math.floor(Math.random() * 100) + 1;
02
03  const guesses = document.querySelector('.guesses’);
04  const lastResult = document.querySelector('.lastResult’);
05  const lowOrHi = document.querySelector('.lowOrHi’);
06
07  const guessSubmit = document.querySelector('.guessSubmit');
08  const guessField = document.querySelector('.guessField’);
09
10  let guessCount = 1;
11  let resetButton;
```

JavaScript

Functions

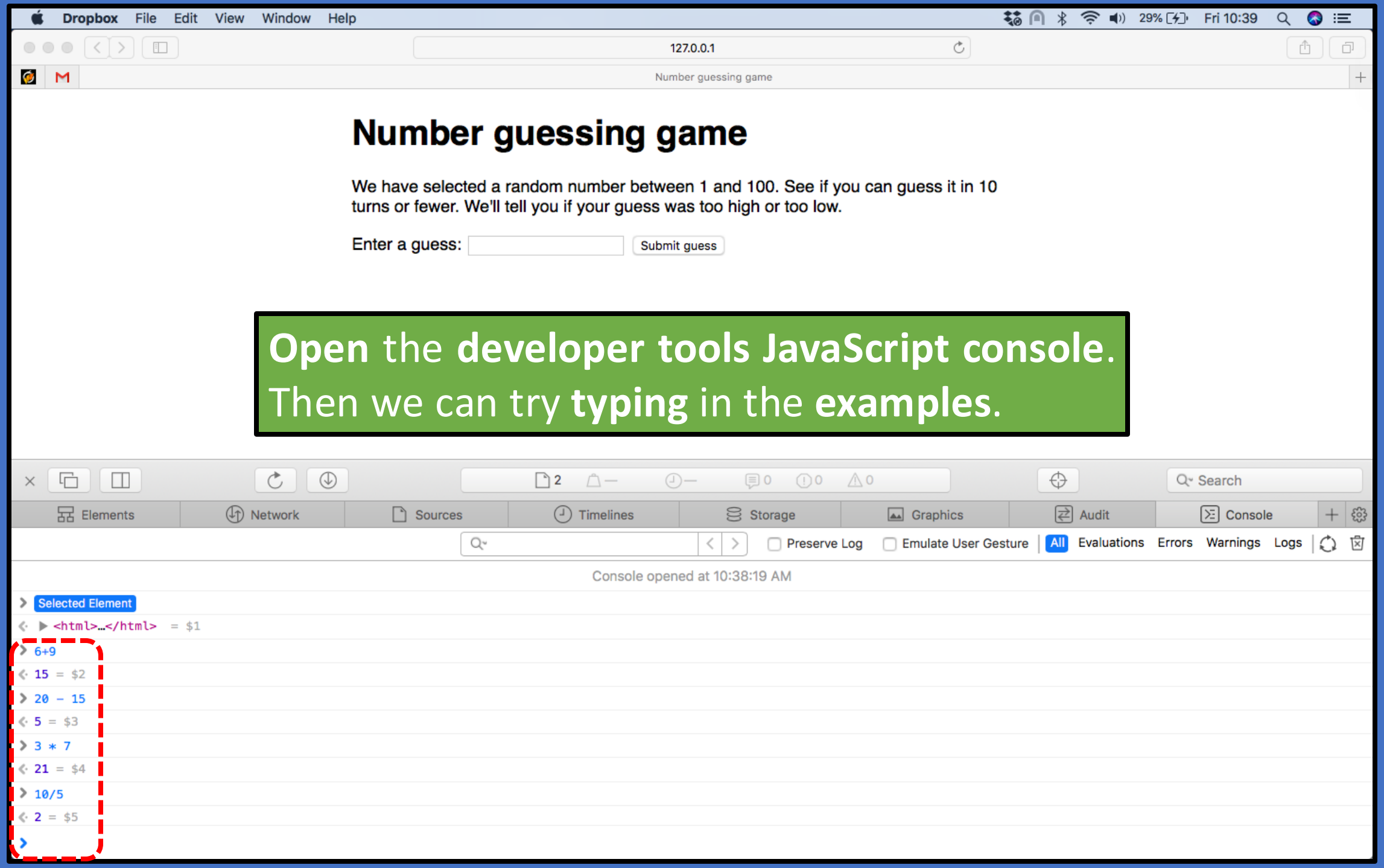

number-guessing-game-start.html
first-splash > number-guessing-game-start.html > html > body > script
36 <input type="submit" value="Submit guess" class="guessSubmit">
37
38
39
40
41
42
43
44
45 <script>
46 let randomNumber = Math.floor(Math.random() * 100) + 1;
47 const guesses = document.querySelector('.guesses');
Add the the checkGuess() function, save your code and refresh the page in your browser.
53 let resetButton;
54
55 function checkGuess() {
56 alert('I am a placeholder');
57 }
58
59
60 </script>
61 </body>
62
63 </html>
Ln 60, Col 12 Spaces: 2 UTF-8 LF HTML Port : 5500

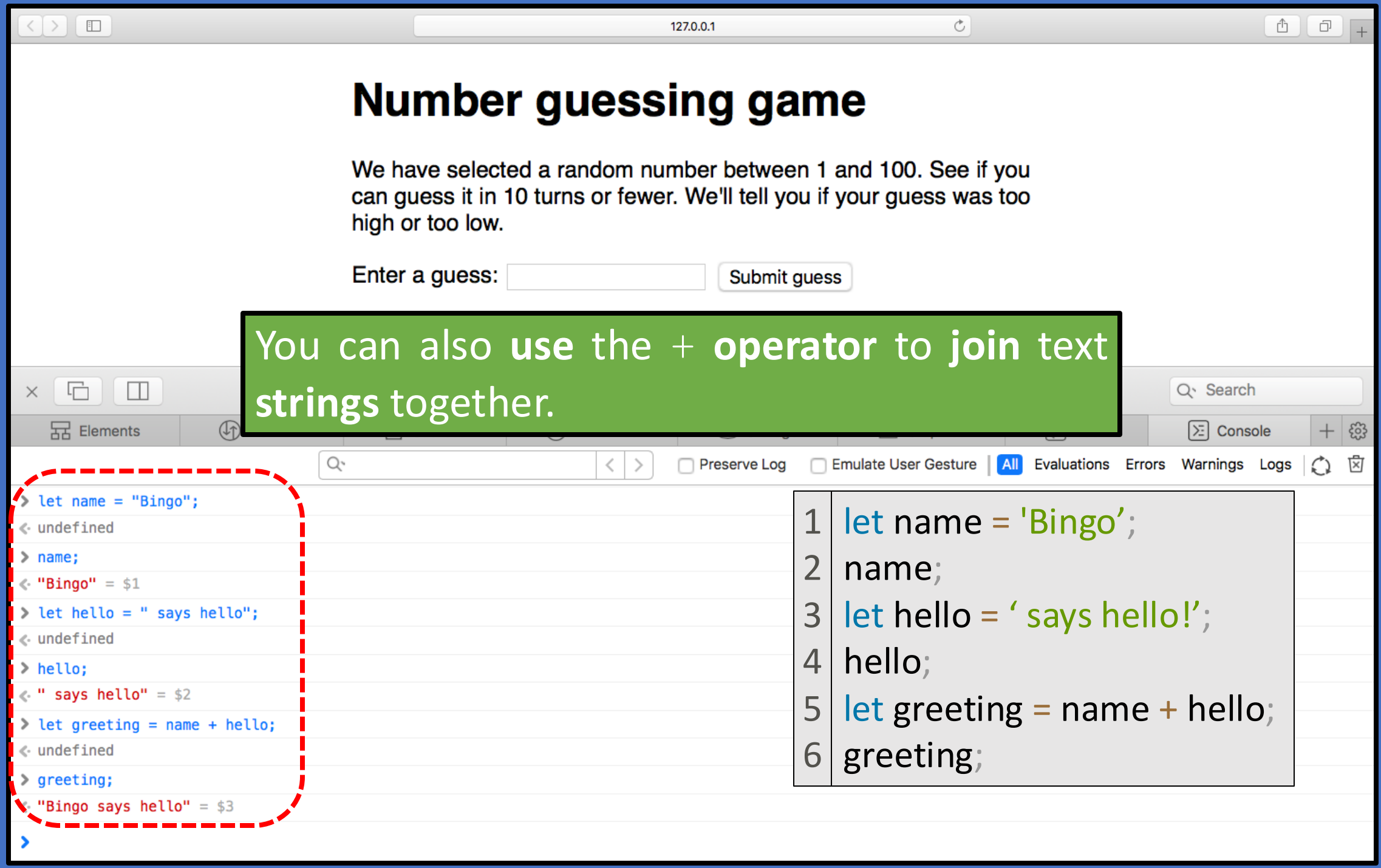

127.0.0.1
Number guessing game
We have selected a random number between 1 and 100. See if you
can guess it i
uess was too
high or too lo
I am a placeholder
Close
Enter a guess.
Go into the developer tools JavaScript console and enter checkGuess( );
Search
Audit
Console
Preserve Log
Emulate User Gesture
All
Evaluations
Errors
Warnings
Logs
Console cleared at 1:15:05 PM
checkGuess();

Operators

Operators

Operator	Name	Example
+	Addition	6 + 9
-	Subtraction	20 - 15
*	Multiplication	3 * 7
/	Division	10 / 5


Dropbox File Edit View Window Help
29% Fri 10:39
127.0.0.1
Number guessing game
Number guessing game
We have selected a random number between 1 and 100. See if you can guess it in 10 turns or fewer. We'll tell you if your guess was too high or too low.
Enter a guess:
Submit guess
Open the developer tools JavaScript console.
Then we can try typing in the examples.
Search
Elements
Network
Sources
Timelines
Storage
Graphics
Audit
Console
Preserve Log
Emulate User Gesture
All
Evaluations
Errors
Warnings
Logs
Console opened at 10:38:19 AM
Selected Element
<html>…</html> = $1
6+9
15 = $2
20 - 15
5 = $3
3 * 7
21 = $4
10/5
2 = $5


127.0.0.1
Number guessing game
We have selected a random number between 1 and 100. See if you can guess it in 10 turns or fewer. We'll tell you if your guess was too high or too low.
Enter a guess:
Submit guess
You can also use the + operator to join text strings together.
Search
Elements
Console
Preserve Log
Emulate User Gesture
All
Evaluations
Errors
Warnings
Logs
let name = "Bingo";
undefined
name;
"Bingo" = $1
let hello = " says hello";
undefined
hello;
" says hello" = $2
let greeting = name + hello;
undefined
greeting;
"Bingo says hello" = $3
1 let name = 'Bingo';
2 name;
3 let hello = ' says hello!';
4 hello;
5 let greeting = name + hello;
6 greeting;

Shortcut operators for assignment

```
1 name += ‘ says hello!';
```

Equivalent to

```
1 name = name + ‘ says hello!';
```

Comparison operators

Operator	Name	Example

`===`	Strict equality (is it exactly the same?)	`5 === 2 + 4 // false`
		`'Chris' === 'Bob' // false`
		`5 === 2 + 3 // true`
		`2 === '2' // false; number versus string`

`!==`	Non-equality (is it not the same?)	`5 !== 2 + 4 // true`
		`'Chris' !== 'Bob' // true`
		`5 !== 2 + 3 // false`
		`2 !== '2' // true; number versus string`

`<`	Less than	`6 < 10 // true` `20 < 10 // false`
`>`	Greater than	`6 > 10 // false` `20 > 10  // true`

Comparison operators

Operator	Name	Example

`===`	Strict equality (is it exactly the same?)	`5 === 2 + 4 // false`
		`'Chris' === 'Bob' // false`
		`5 === 2 + 3 // true`
		`2 === '2' // false; number versus string`

`!==`	Non-equality (is it not the same?)	`5 !== 2 + 4 // true`
		`'Chris' !== 'Bob' // true`
		`5 !== 2 + 3 // false`
		`2 !== '2' // true; number versus string`

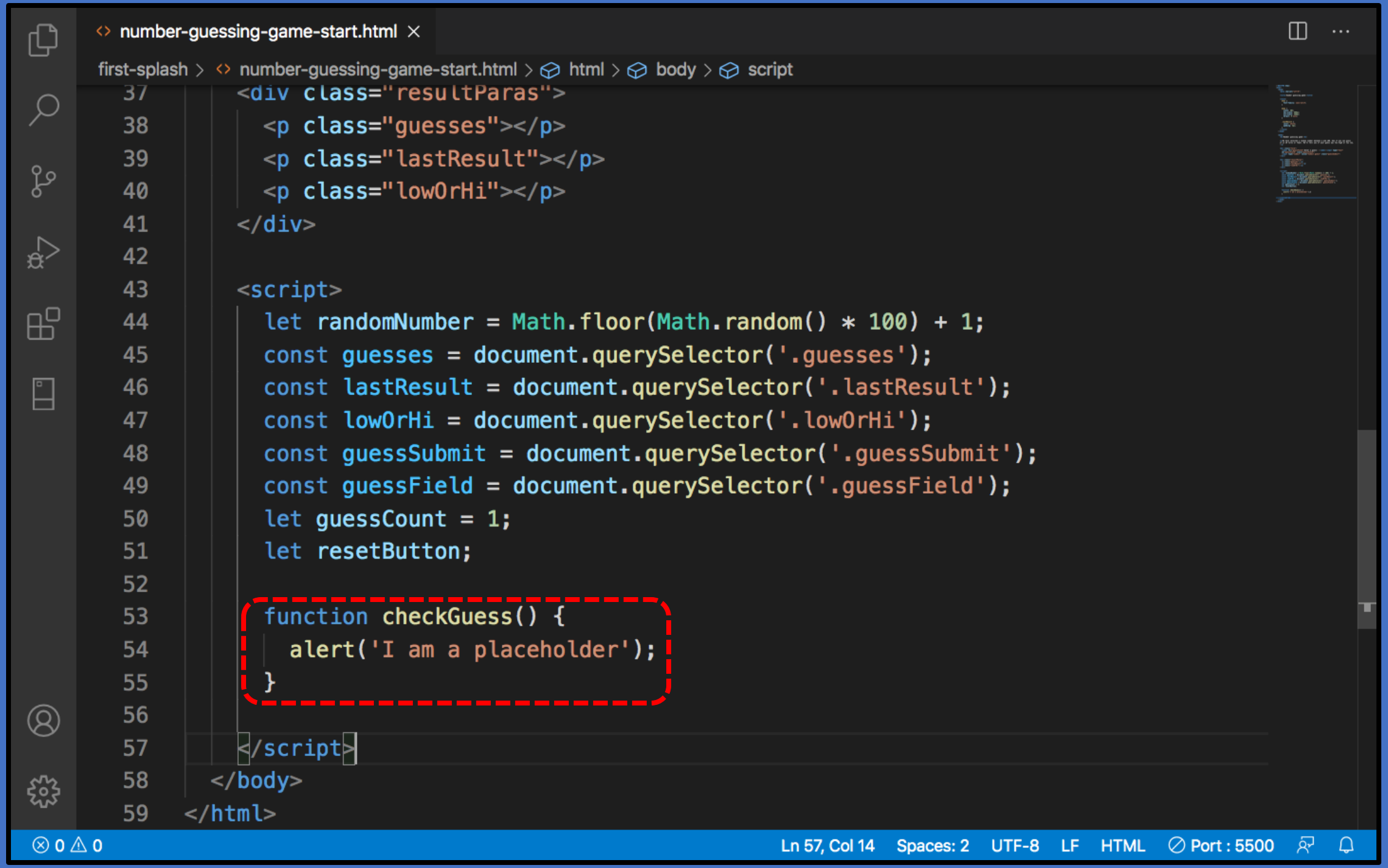
`<`	Less than	`6 < 10 // true` `20 < 10 // false`
`>`	Greater than	`6 > 10 // false` `20 > 10  // true`

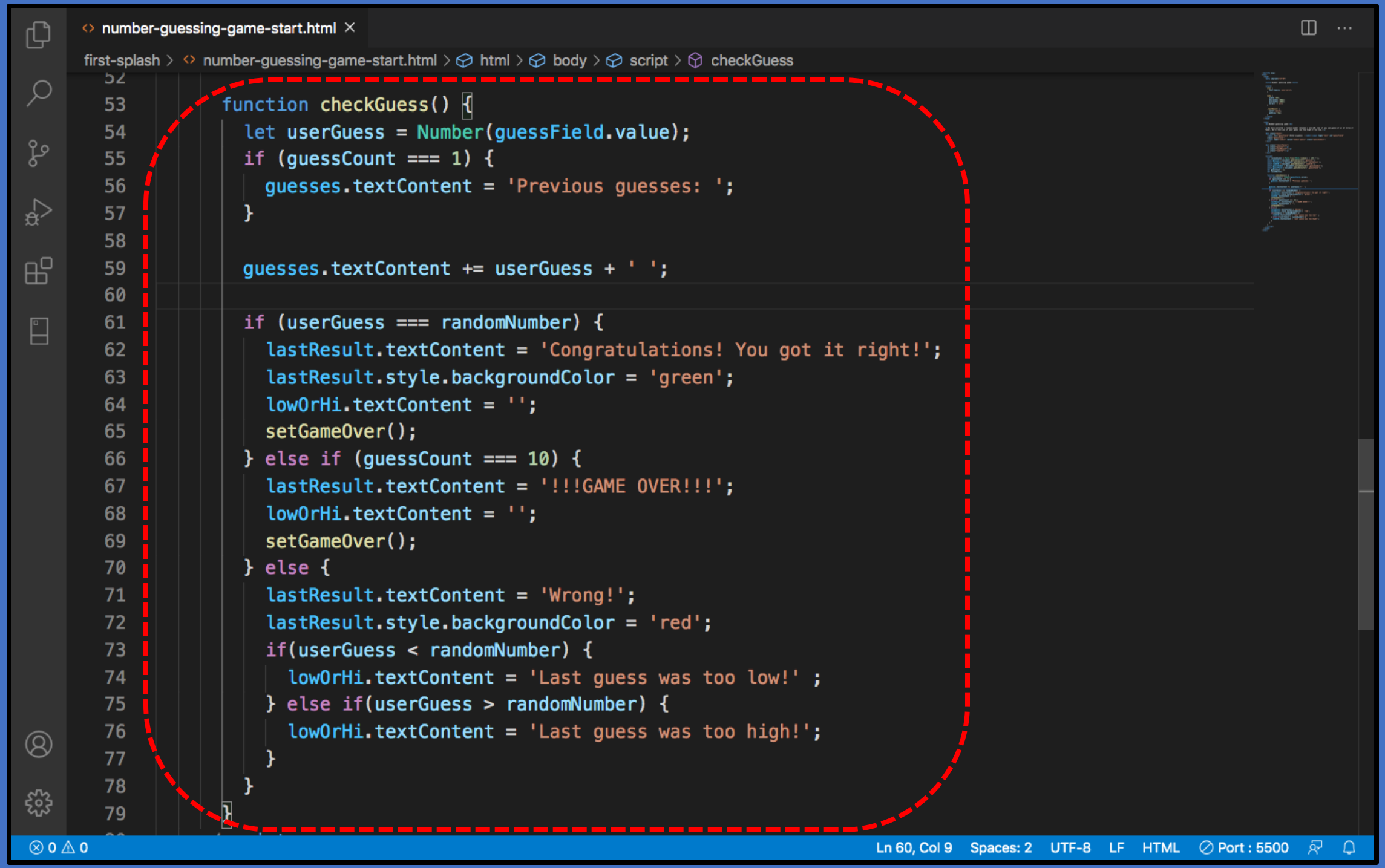
Functions again ☺

Conditionals

Replace your current `checkGuess()` **function** with this version instead.

```
01  function checkGuess() {
02    let userGuess = Number(guessField.value);
03    if (guessCount === 1) {
04      guesses.textContent = 'Previous guesses: ‘;
05    }
06    guesses.textContent += userGuess + ' ‘;
07
08    if (userGuess === randomNumber) {
09      lastResult.textContent = 'Congratulations! You got it right!’;
10      lastResult.style.backgroundColor = 'green’;
11      lowOrHi.textContent = ’’;
12      setGameOver();
13    } else if (guessCount === 10) {
14      lastResult.textContent = '!!!GAME OVER!!!’;
15      setGameOver();
16    } else {
17      lastResult.textContent = 'Wrong!’;
18      lastResult.style.backgroundColor = 'red’;
19      if(userGuess < randomNumber) {
20          lowOrHi.textContent = 'Last guess was too low!'; }
21      else if(userGuess > randomNumber) {
22          lowOrHi.textContent = 'Last guess was too high!'; }
23      }
24    }
25
26      guessCount++;
27      guessField.value = ’’;
28      guessField.focus();
29  }
```


number-guessing-game-start.html
first-splash > number-guessing-game-start.html > html > body > script
37
38
39
40
41
42
43 <script>
44 let randomNumber = Math.floor(Math.random() * 100) + 1;
45 const guesses = document.querySelector('.guesses');
46 const lastResult = document.querySelector('.lastResult');
47 const lowOrHi = document.querySelector('.lowOrHi');
48 const guessSubmit = document.querySelector('.guessSubmit');
49 const guessField = document.querySelector('.guessField');
50 let guessCount = 1;
51 let resetButton;
52
53 function checkGuess() {
54 alert('I am a placeholder');
55 }
56
57 </script>
58 </body>
59 </html>
0 0
Ln 57, Col 14
Spaces: 2
UTF-8
LF
HTML
Port : 5500


number-guessing-game-start.html
first-splash > number-guessing-game-start.html > html > body > script > checkGuess
function checkGuess() {
let userGuess = Number(guessField.value);
if (guessCount === 1) {
guesses.textContent = 'Previous guesses: ';
}
guesses.textContent += userGuess + ' ';
if (userGuess === randomNumber) {
lastResult.textContent = 'Congratulations! You got it right!';
lastResult.style.backgroundColor = 'green';
lowOrHi.textContent = '';
setGameOver();
} else if (guessCount === 10) {
lastResult.textContent = '!!!GAME OVER!!!';
lowOrHi.textContent = '';
setGameOver();
} else {
lastResult.textContent = 'Wrong!';
lastResult.style.backgroundColor = 'red';
if(userGuess < randomNumber) {
lowOrHi.textContent = 'Last guess was too low!' ;
} else if(userGuess > randomNumber) {
lowOrHi.textContent = 'Last guess was too high!';
}
}
}
0 0
Ln 60, Col 9
Spaces: 2
UTF-8
LF
HTML
Port : 5500

We **included** the **value** entered in the **guess field** in the `Number()` **constructor**, just to make sure the value is definitely a **number**.



```
01  function checkGuess() {
02    let userGuess = Number(guessField.value);
03    if (guessCount === 1) {
04      guesses.textContent = 'Previous guesses: ';
05    }
06    guesses.textContent += userGuess + ' ';
07
08    if (userGuess === randomNumber) {
09      lastResult.textContent = 'Congratulations! You got it right!';
10      lastResult.style.backgroundColor = 'green';
11      lowOrHi.textContent = '';
12      setGameOver();
13    } else if (guessCount === 10) {
14      lastResult.textContent = '!!!GAME OVER!!!';
15      setGameOver();
16    } else {
17      lastResult.textContent = 'Wrong!';
18      lastResult.style.backgroundColor = 'red';
19      if(userGuess < randomNumber) {
20          lowOrHi.textContent = 'Last guess was too low!'; }
21      else if(userGuess > randomNumber) {
22          lowOrHi.textContent = 'Last guess was too high!'; }
23      }
24    }
25
26      guessCount++;
27      guessField.value = '';
28      guessField.focus();
29  }
```

Is the **player's first go** or **not**?

```
01  function checkGuess() {
02    let userGuess = Number(guessField.value);
03    if (guessCount === 1) {
04      guesses.textContent = 'Previous guesses: ';
05    }
06    guesses.textContent += userGuess + ' ';
07
08    if (userGuess === randomNumber) {
09      lastResult.textContent = 'Congratulations! You got it right!';
10      lastResult.style.backgroundColor = 'green';
11      lowOrHi.textContent = '';
12      setGameOver();
13    } else if (guessCount === 10) {
14      lastResult.textContent = '!!!GAME OVER!!!';
15      setGameOver();
20          lowOrHi.textContent = 'Last guess was too low!'; }
21      else if(userGuess > randomNumber) {
22          lowOrHi.textContent = 'Last guess was too high!'; }
23      }
24    }
25
26      guessCount++;
27      guessField.value = '';
28      guessField.focus();
29  }
```

We are going to **display** "Previous guesses" in the **output** only if it is the **user first guess**.

Appends the current **userGuess value** onto the **end** of the **guesses paragraph, plus** a **blank space** so there will be a **space between each guess** shown.

```
01  function checkGuess() {
02    let userGuess = Number(guessField.value);
03    if (guessCount === 1) {
04      guesses.textContent = 'Previous guesses: ‘;
05    }
06    guesses.textContent += userGuess + ' ';
07
08    if (userGuess === randomNumber) {
09      lastResult.textContent = 'Congratulations! You got it right!’;
10      lastResult.style.backgroundColor = 'green’;
11      lowOrHi.textContent = ’’;
12      setGameOver();
13    } else if (guessCount === 10) {
14      lastResult.textContent = '!!!GAME OVER!!!’;
15      setGameOver();
16    } else {
17      lastResult.textContent = 'Wrong!’;
18      lastResult.style.backgroundColor = 'red’;
19      if(userGuess < randomNumber) {
20          lowOrHi.textContent = 'Last guess was too low!'; }
21      else if(userGuess > randomNumber) {
22          lowOrHi.textContent = 'Last guess was too high!'; }
23      }
24    }
25
26      guessCount++;
27      guessField.value = ’’;
28      guessField.focus();
29  }
```

```
<script>
  let randomNumber =
  Math.floor(Math.random() * 100) + 1;
  …
</script>
```

`randomNumber` set at the top of our **JavaScript**.

```
function checkGuess() {
  let userGuess = Number(guessField.value);
  if (guessCount === 1) {
    guesses.textContent = 'Previous guesses: ‘;
  }
  guesses.textContent += userGuess + ' ‘;

  if (userGuess === randomNumber) {
    lastResult.textContent = 'Congratulations! You got it right!’;
    lastResult.style.backgroundColor = 'green’;
    lowOrHi.textContent = ’’;
    setGameOver();
  } else if (guessCount === 10) {
    lastResult.textContent = '!!!GAME OVER!!!’;
    setGameOver();
  } else {
    lastResult.textContent = 'Wrong!’;
    lastResult.style.backgroundColor = 'red’;
    if(userGuess < randomNumber) {
        lowOrHi.textContent = 'Last guess was too low!'; }
    else if(userGuess > randomNumber) {
        lowOrHi.textContent = 'Last guess was too high!'; }
    }
  }

    guessCount++;
    guessField.value = ’’;
    guessField.focus();
}
```

clear the **contents** of the **Low/High guess information box**, and **run** a **function** called **setGameOver()**, which we'll **discuss later**.

```
01  function checkGuess() {
02    let userGuess = Number(guessField.value);
03    if (guessCount === 1) {
04      guesses.textContent = 'Previous guesses: ‘;
05    }
06    guesses.textContent += userGuess + ' ‘;
07
08    if (userGuess === randomNumber) {
09      lastResult.textContent = 'Congratulations! You got it right!’;
10      lastResult.style.backgroundColor = 'green’;
11      lowOrHi.textContent = '';
12      setGameOver();
13    } else if (guessCount === 10) {
14      lastResult.textContent = '!!!GAME OVER!!!’;
15      setGameOver();
16    } else {
17      lastResult.textContent = 'Wrong!’;
18      lastResult.style.backgroundColor = 'red’;
19      if(userGuess < randomNumber) {
20          lowOrHi.textContent = 'Last guess was too low!'; }
21      else if(userGuess > randomNumber) {
22          lowOrHi.textContent = 'Last guess was too high!'; }
23      }
24    }
25
26      guessCount++;
27      guessField.value = '';
28      guessField.focus();
29  }
```

Increment the **number** of **attempts**, **empty** the **value** out of the **form text field** and **focus** it **again**, ready for the **next guess** to be entered.

```
01  function checkGuess() {
02    let userGuess = Number(guessField.value);
03    if (guessCount === 1) {
04      guesses.textContent = 'Previous guesses: ‘;
05    }
06    guesses.textContent += userGuess + ' ‘;
07
08    if (userGuess === randomNumber) {
09      lastResult.textContent = 'Congratulations! You got it right!’;
10      lastResult.style.backgroundColor = 'green’;
11      lowOrHi.textContent = '';
12      setGameOver();
13    } else if (guessCount === 10) {
14      lastResult.textContent = '!!!GAME OVER!!!’;
15      setGameOver();
16    } else {
17      lastResult.textContent = 'Wrong!’;
18      lastResult.style.backgroundColor = 'red’;
19      if(userGuess < randomNumber) {
20          lowOrHi.textContent = 'Last guess was too low!'; }
21      else if(userGuess > randomNumber) {
22          lowOrHi.textContent = 'Last guess was too high!'; }
23      }
24    }
25
26      guessCount++;
27      guessField.value = '';
28      guessField.focus();
29  }
```

Events

Events

- We want to **call** the `checkGuess()` **function** when the **"Submit guess" button** is **pressed.**
  - ✓To do this we **need** to **use** an **event**.
- **Events** are **things** that **happen** in the **browser** in **response** to which we can **run blocks** of **code**.
  - ✓A **button** being **clicked**, a **page loading**, a **video playing**, etc.

Event listener & event handler

- The **constructs** that **listen** out for the **event happening** are called **event listeners**.
  - ✓The **blocks of code** that **run** in **response** to the **event firing** are called **event handlers**.

Event listener & event handler

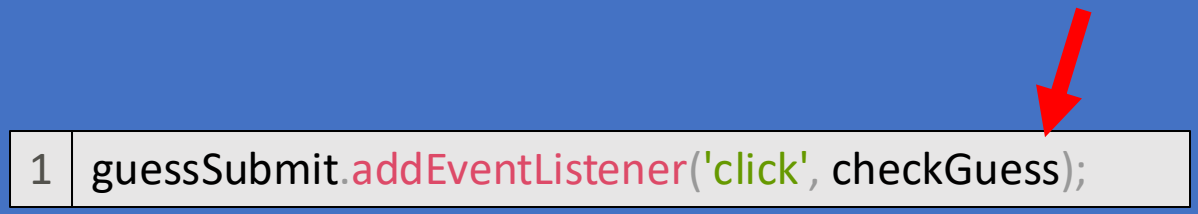
- We are **adding** an **event listener** to the `guessSubmit` **button**.

```
1  guessSubmit.addEventListener('click', checkGuess);
```

- `addEventListener` is a **method** that **takes two arguments**.
  - ✓The **type of event** we are **listening** out for as a **string.**
    - In this case **click.**
  - ✓The **code** we **want** to **run when** the **event occurs**
    - In this case the `checkGuess()` **function**.

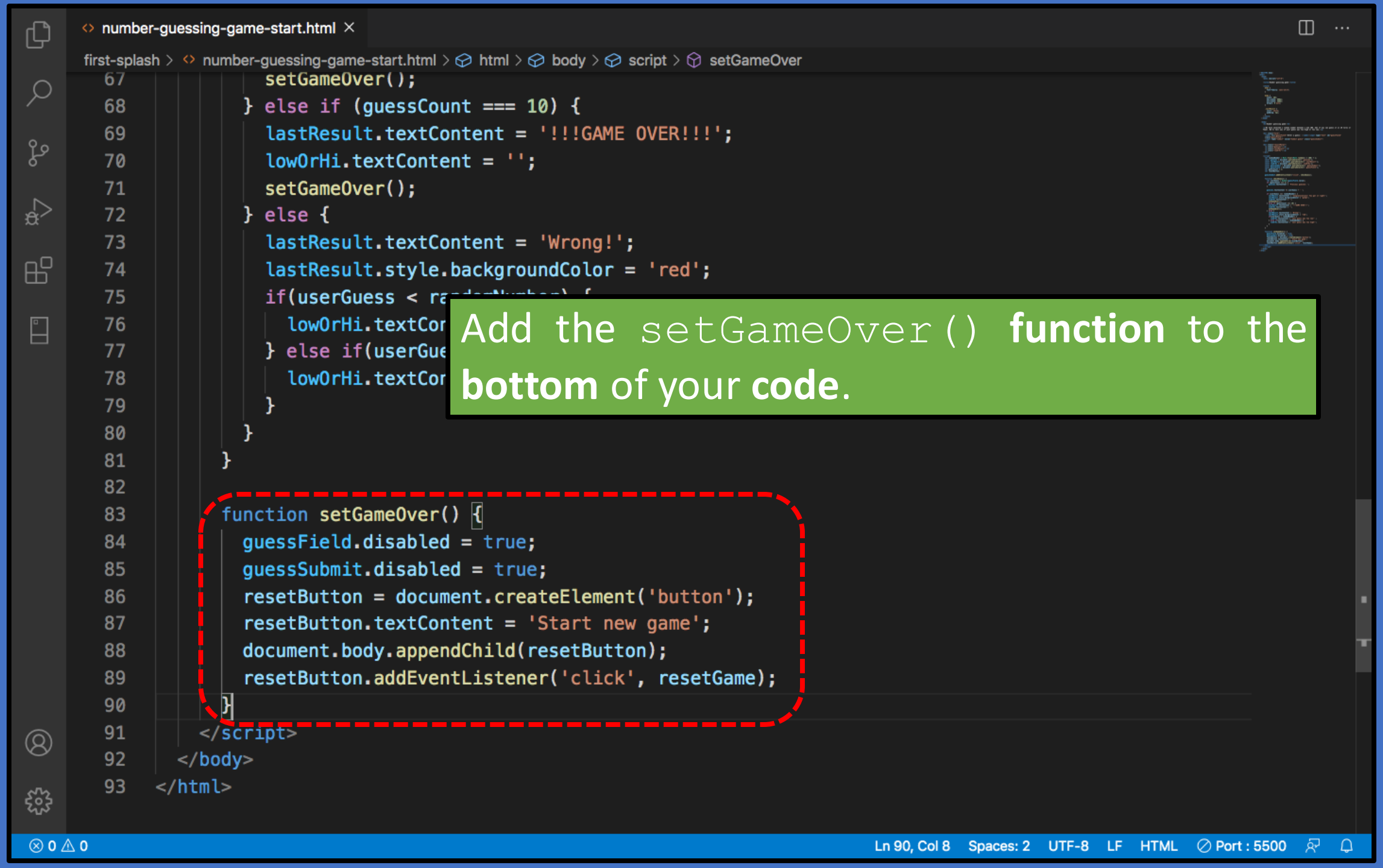
Event listener & event handler

- Note that **we don't need to specify** the **parentheses** when **writing it inside `addEventListener()`.**



```
1  guessSubmit.addEventListener('click', checkGuess);
```

Finish the game functionality


number-guessing-game-start.html
first-splash > number-guessing-game-start.html > html > body > script > setGameOver
setGameOver();
} else if (guessCount === 10) {
lastResult.textContent = '!!!GAME OVER!!!';
lowOrHi.textContent = '';
setGameOver();
} else {
lastResult.textContent = 'Wrong!';
lastResult.style.backgroundColor = 'red';
if(userGuess <
lowOrHi.textCor
} else if(userGue
lowOrHi.textCor
}
}
}
function setGameOver() {
guessField.disabled = true;
guessSubmit.disabled = true;
resetButton = document.createElement('button');
resetButton.textContent = 'Start new game';
document.body.appendChild(resetButton);
resetButton.addEventListener('click', resetGame);
}
</script>
</body>
</html>
Add the setGameOver() function to the bottom of your code.
0 0
Ln 90, Col 8
Spaces: 2
UTF-8
LF
HTML
Port : 5500

Finishing the game functionality

- Add the **following code**, again to the **bottom of your JavaScript**

```
01  function resetGame() {
02    guessCount = 1;
03
04    const resetParas = document.querySelectorAll('.resultParas p’);
05    for (let i = 0 ; i < resetParas.length ; i++) {
06      resetParas[i].textContent = ’’;
07    }
08
09    resetButton.parentNode.removeChild(resetButton);
10
11     guessField.disabled = false;
12     guessSubmit.disabled = false;
13     guessField.value = ’’;
14     guessField.focus();
15
16     lastResult.style.backgroundColor = 'white’;
17
18     randomNumber = Math.floor(Math.random() * 100) + 1;
19  }
```

number-guessing-game-start.html

first-splash > number-guessing-game-start.html > html > body > script > resetGame

```
82
83      function setGameOver() {
84        guessField.disabled = true;
85        guessSubmit.disabled = true;
86        resetButton = document.createElement('button');
87        resetButton.textContent = 'Start new game';
88        document.body.appendChild(resetButton);
89        resetButton.addEventListener('click', resetGame);
90      }
91
92      function resetGame() {
93        guessCount = 1;
94        const resetParas = document.querySelectorAll('.resultParas p');
95        for(let i = 0 ; i < resetParas.length ; i++) {
96          resetParas[i].textContent = '';
97        }
98
99        resetButton.parentNode.removeChild(resetButton);
100       guessField.disabled = false;
101       guessSubmit.disabled = false;
102       guessField.value = '';
103       guessField.focus();
104       lastResult.style.backgroundColor = 'white';
105       randomNumber = Math.floor(Math.random() * 100) + 1;
106     }
107   </script>
108 </body>
109 </html>
```

```
1 <div class="resultParas">
2   <p class="guesses"></p>
3   <p class="lastResult"></p>
4   <p class="lowOrHi"></p>
5 </div>
```

This **function resets** everything to how it was at the **start of the game**.

0 ⚠ 0 Ln 92, Col 29 Spaces: 2 UTF-8 LF HTML Port : 5500

Finishing the game functionality

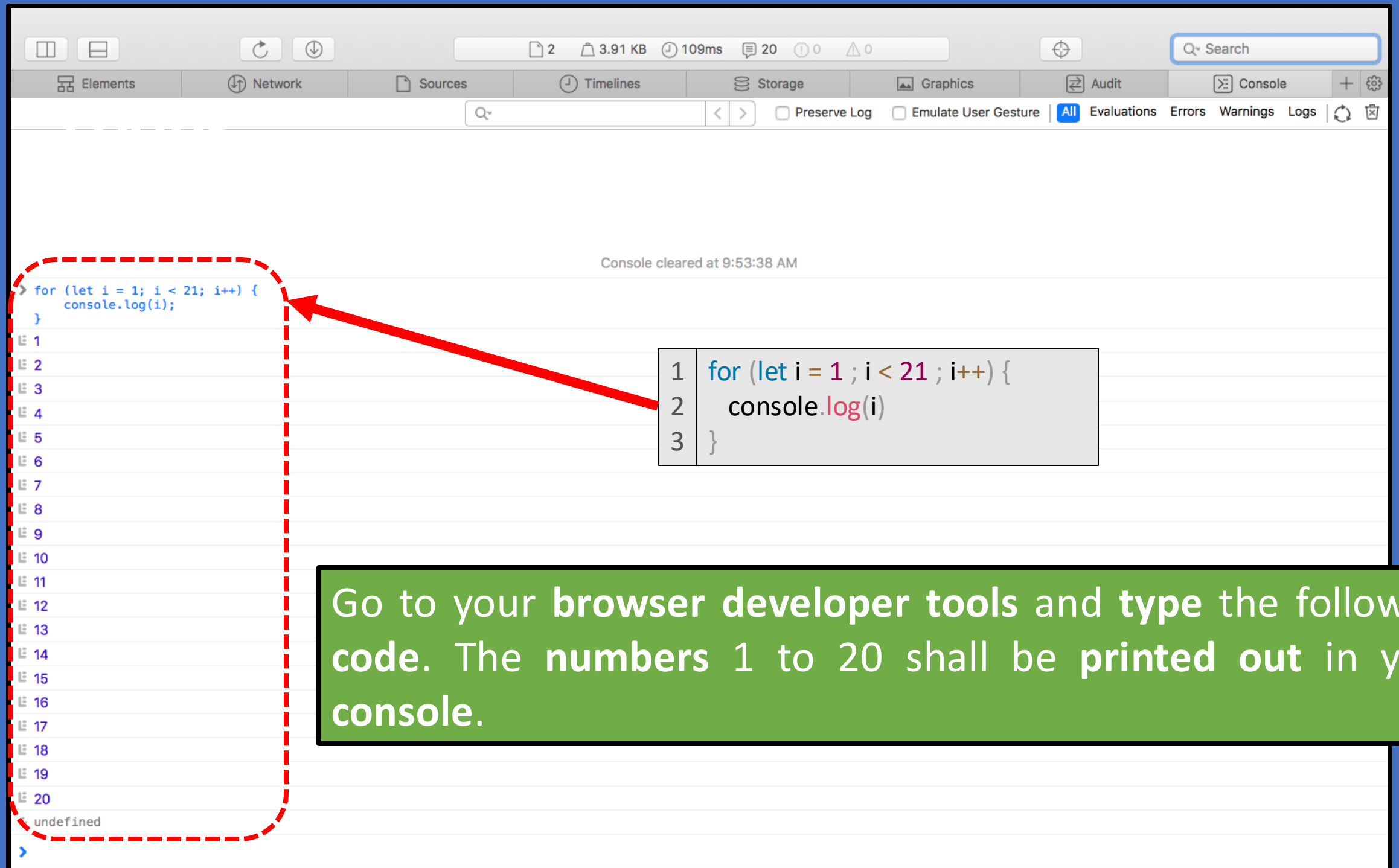
- Lines **04-07 empty** all the text.
  - ✓We **select** all **paragraphs inside**

```
<div class="resultParas">
  <p>…</p>
  <p>…</p>
  <p>…</p>
</div>
```

  - ✓Then **loop** through **each** one, **setting** their `textContent` to an empty string.

```
01  function resetGame() {
02    guessCount = 1;
03
04    const resetParas = document.querySelectorAll('.resultParas p');
05    for (let i = 0 ; i < resetParas.length ; i++) {
06      resetParas[i].textContent = '';
07    }
08
09    resetButton.parentNode.removeChild(resetButton);
10
11    guessField.disabled = false;
12    guessSubmit.disabled = false;
13    guessField.value = '';
14    guessField.focus();
15
16    lastResult.style.backgroundColor = 'white';
17
18    randomNumber = Math.floor(Math.random() * 100) + 1;
19  }
```

Loops


2
3.91 KB
109ms
20
0
0
Search
Elements
Network
Sources
Timelines
Storage
Graphics
Audit
Console
Preserve Log
Emulate User Gesture
All
Evaluations
Errors
Warnings
Logs
Console cleared at 9:53:38 AM
for (let i = 1; i < 21; i++) {
console.log(i);
}
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
undefined
1 for (let i = 1 ; i < 21 ; i++) {
2 console.log(i)
3 }
Go to your **browser developer tools** and **type** the following **code**. The **numbers** 1 to 20 shall be **printed out** in your **console**.

number-guessing-game-start.html

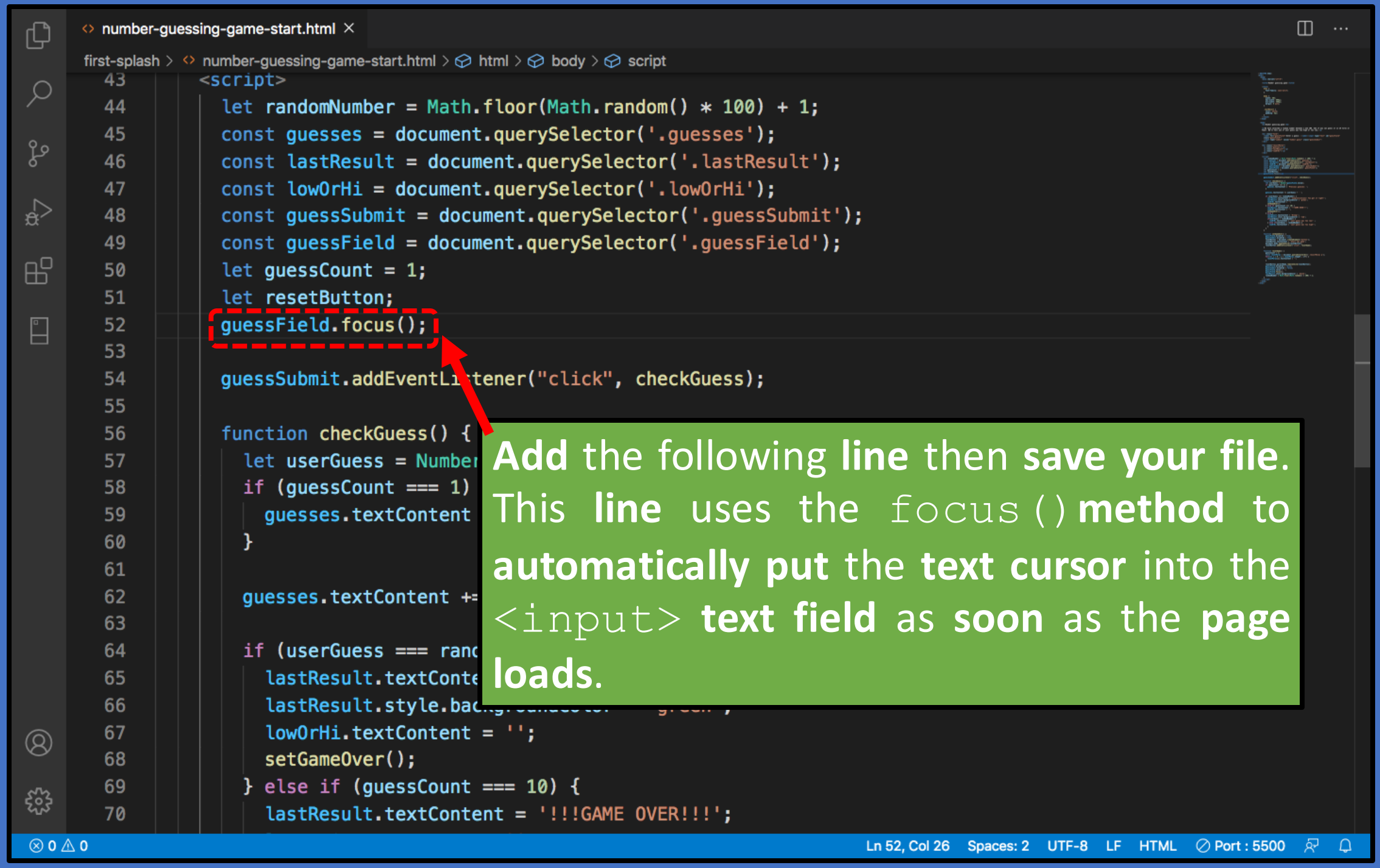
first-splash > number-guessing-game-start.html > html > body > script > resetGame

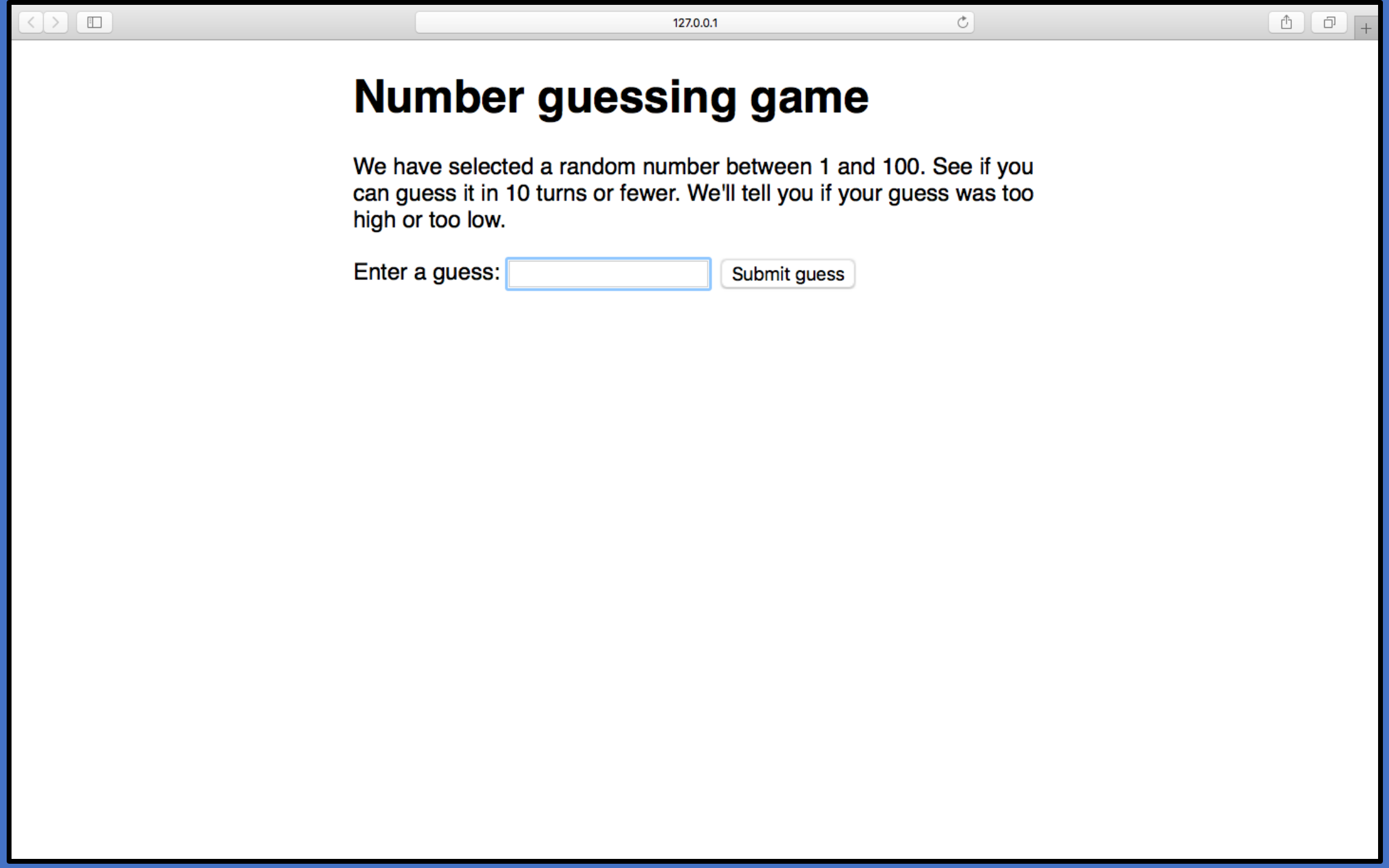
```
82
83      function setGameOver() {
84        guessField.disabled = true;
85        guessSubmit.disabled = true;
86        resetButton = document.createElement('button');
87        resetButton.textContent = 'Start new game';
88        document.body.appendChild(resetButton);
89        resetButton.addEventListener('click', resetGame);
90      }
91
92      function resetGame() {
93        guessCount = 1;
94        const resetParas = document.querySelectorAll('.resultParas p');
95        for(let i = 0 ; i < resetParas.length ; i++) {
96          resetParas[i].textContent = '';
97        }
98
99        resetButton.parentNode.removeChild(resetButton);
100
101
102
103
104
105
106
107
108   </body>
109 </html>
```

```
1 <div class="resultParas">
2   <p class="guesses"></p>
3   <p class="lastResult"></p>
4   <p class="lowOrHi"></p>
5 </div>
```

This **code creates** a **variable** containing a **list** of **all** the **paragraphs inside** `<div class="resultParas">` using the `querySelectorAll()` **method**, then it **loops** through **each** one, **removing** the **text content** of each.

A **small discussion** on **objects**


number-guessing-game-start.html
first-splash > number-guessing-game-start.html > html > body > script
<script>
let randomNumber = Math.floor(Math.random() * 100) + 1;
const guesses = document.querySelector('.guesses');
const lastResult = document.querySelector('.lastResult');
const lowOrHi = document.querySelector('.lowOrHi');
const guessSubmit = document.querySelector('.guessSubmit');
const guessField = document.querySelector('.guessField');
let guessCount = 1;
let resetButton;
guessField.focus();
guessSubmit.addEventListener("click", checkGuess);
function checkGuess() {
let userGuess = Number
if (guessCount === 1)
guesses.textContent
}
guesses.textContent +=
if (userGuess === ranc
lastResult.textConte
lowOrHi.textContent = '';
setGameOver();
} else if (guessCount === 10) {
lastResult.textContent = '!!!GAME OVER!!!';
Add the following line then save your file. This line uses the focus() method to automatically put the text cursor into the <input> text field as soon as the page loads.
Ln 52, Col 26
Spaces: 2
UTF-8
LF
HTML
Port : 5500


127.0.0.1
Number guessing game
We have selected a random number between 1 and 100. See if you can guess it in 10 turns or fewer. We'll tell you if your guess was too high or too low.
Enter a guess:
Submit guess

A small discussion on objects

- In **JavaScript**, **everything** is an **object**.
- An **object** is a **collection** of **related functionality stored** in a **single grouping**.
- **For now**, we'll just **briefly discuss** the **built-in objects** that your **browser contains**.

```
1 guessField.focus();
```

A small discussion on objects

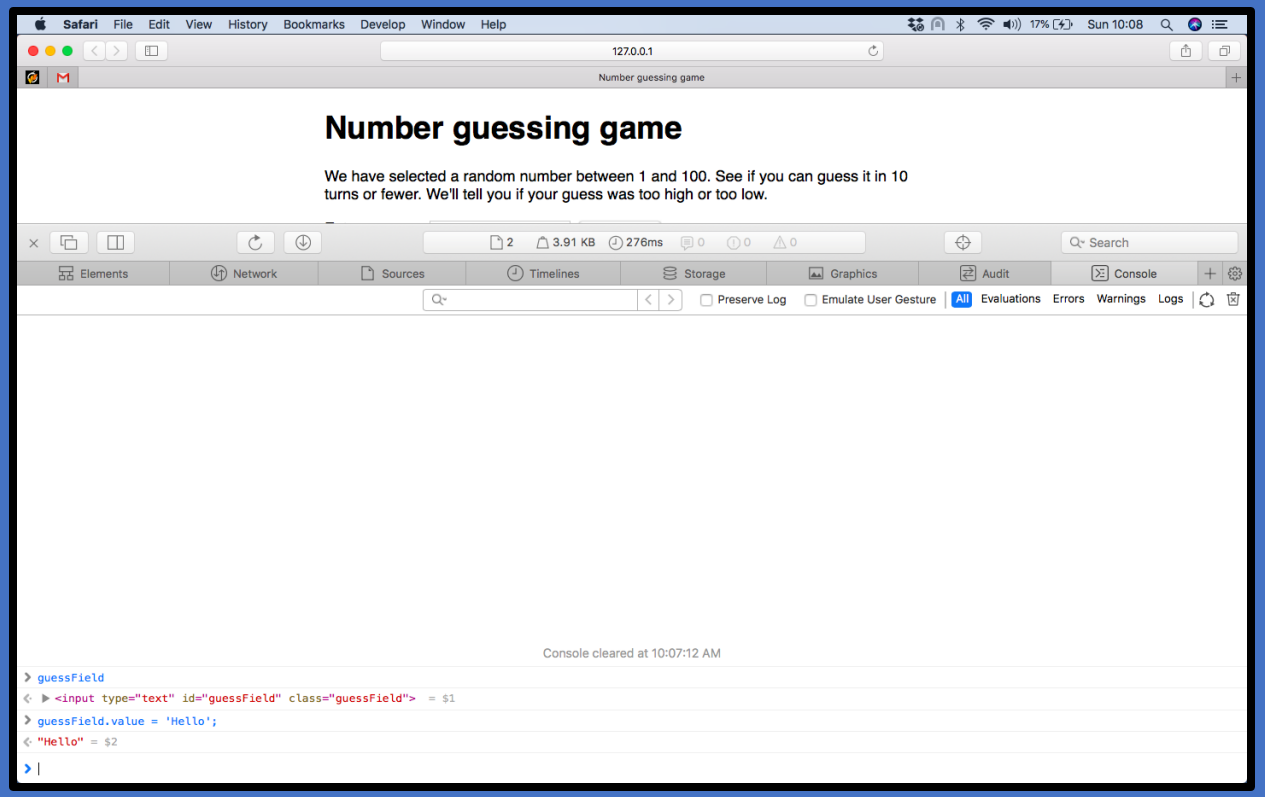
- We first created a `guessField` **constant** that **stores** a **reference** to the **text input form field** in our **HTML**.
- To get this **reference**, we used the `querySelector()` **method** of the **document object**.
  - ✓`querySelector()` receives a **CSS selector** that **selects** the **element** you **want** a **reference to**.
- **One method available** to **input elements** is `focus()`.

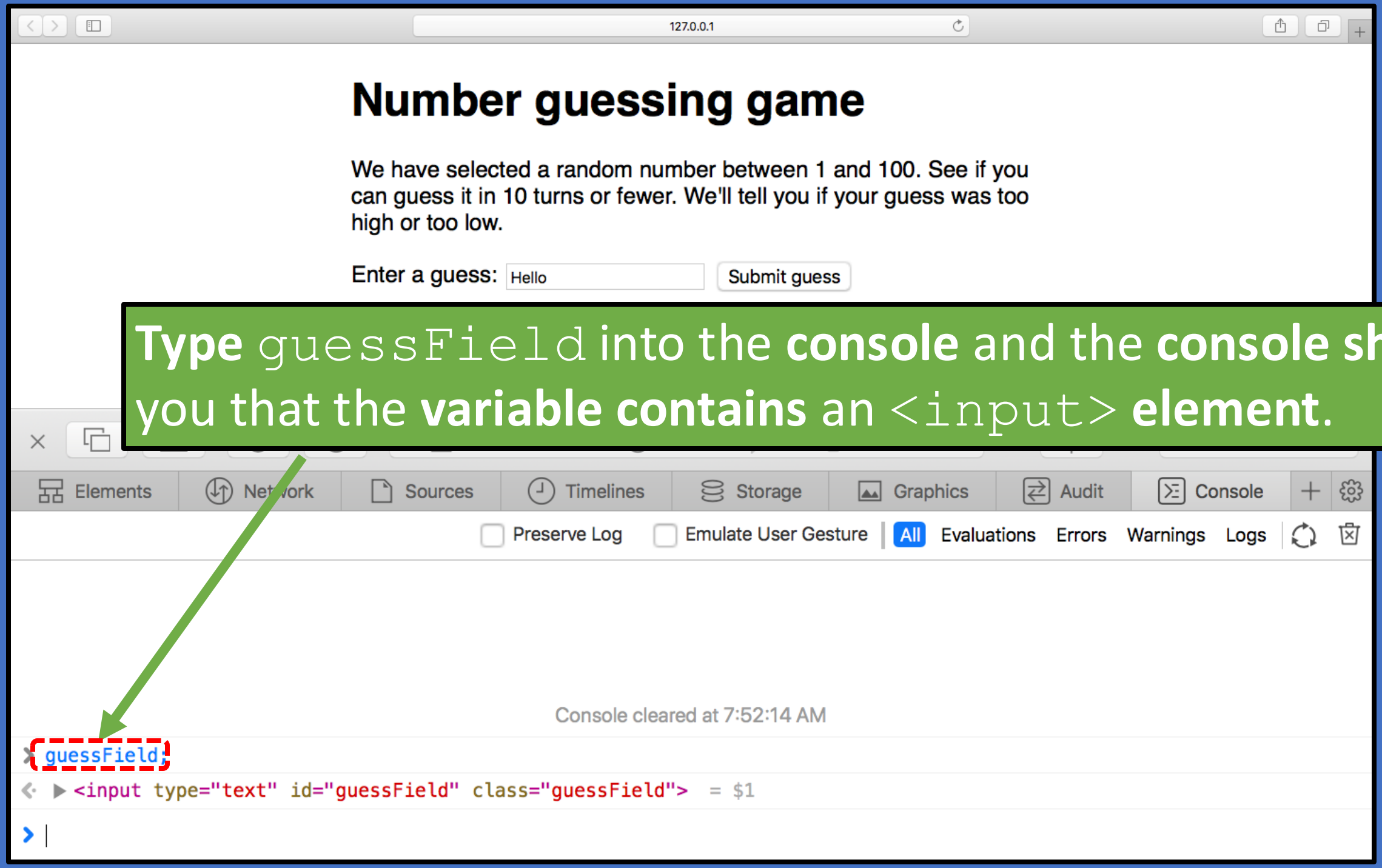
```
1 const guessField = document.querySelector('.guessField');
```

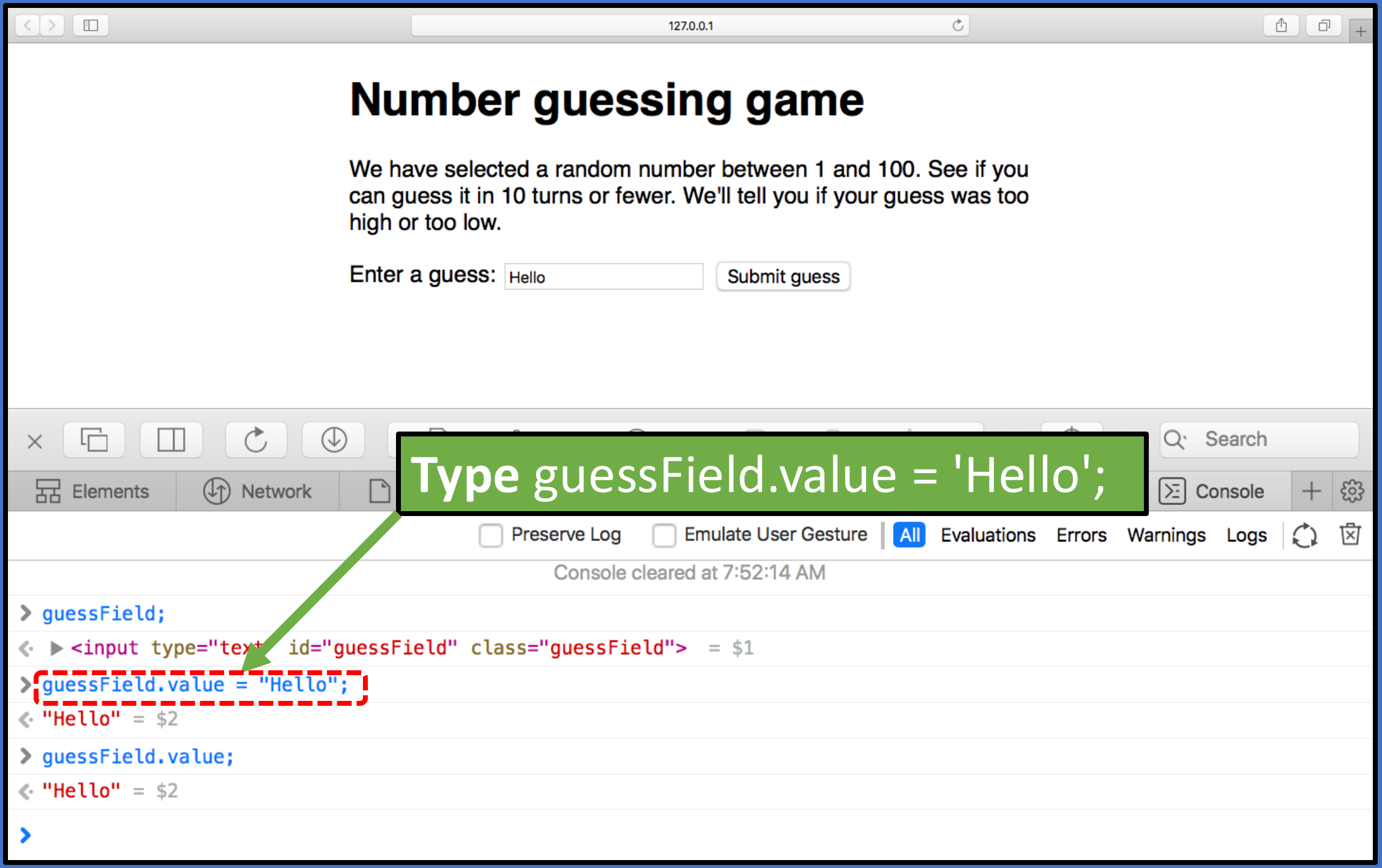
```
1 guessField.focus();
```

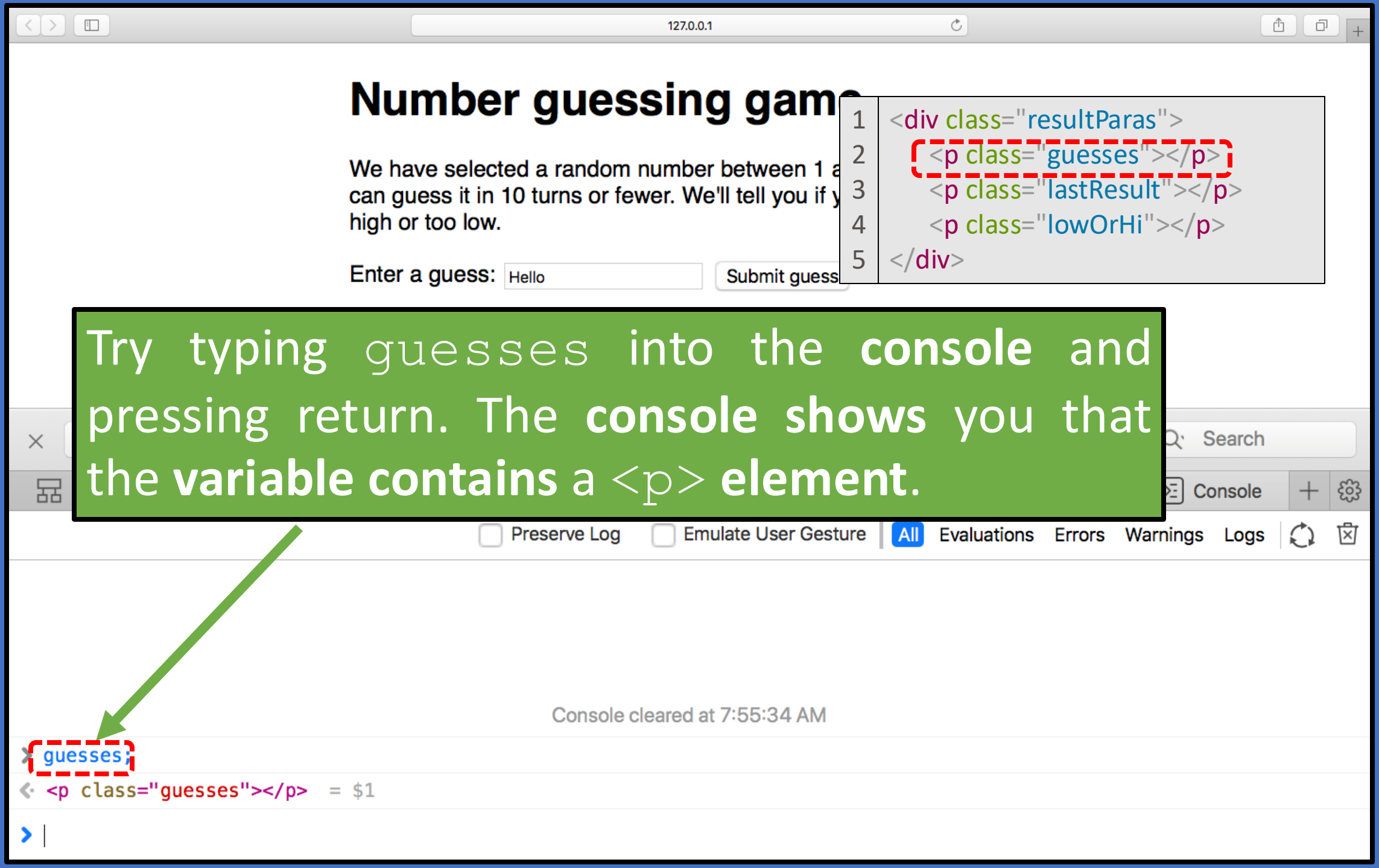
Playing with browser objects

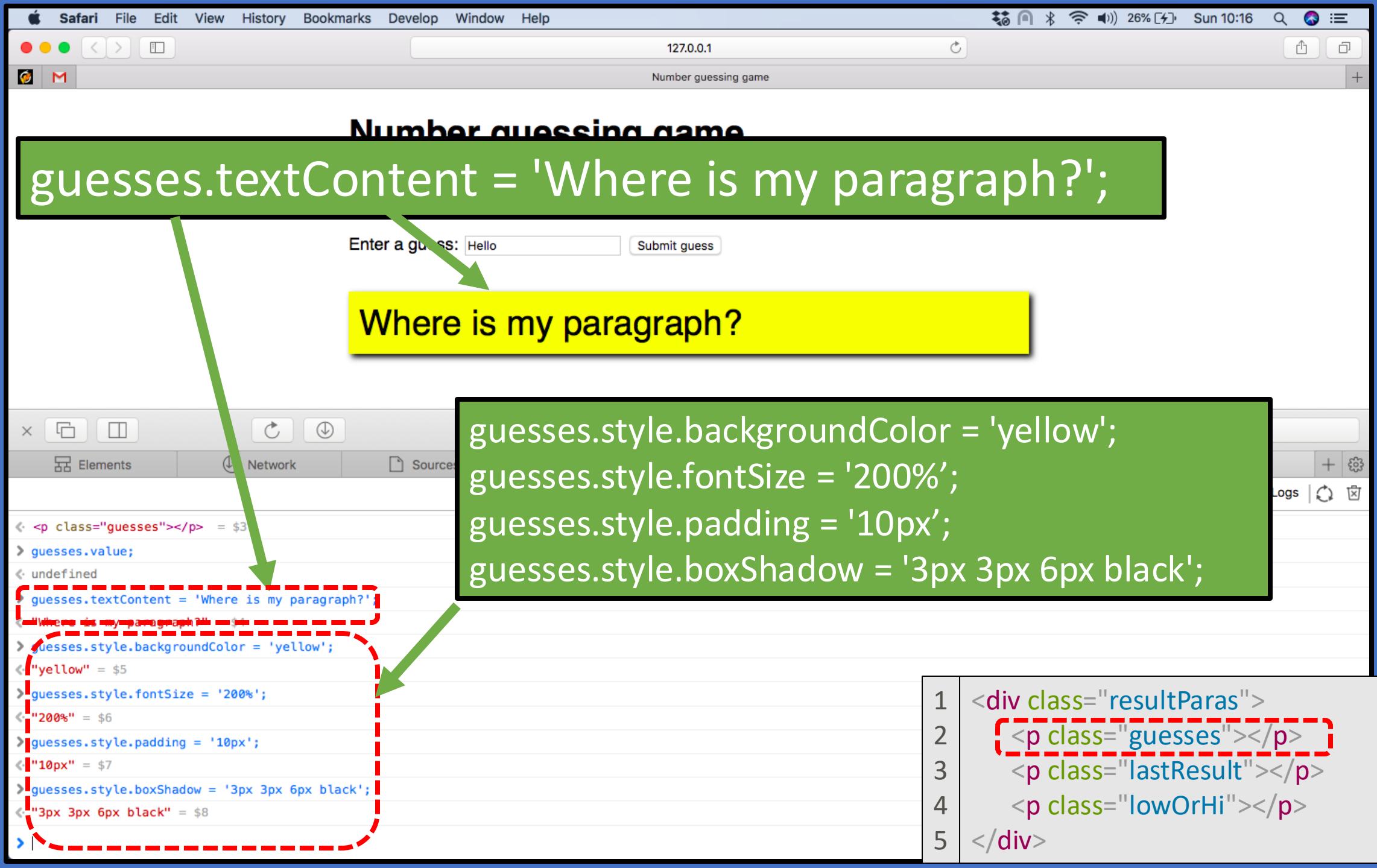
1. **Open up** your **program** in a **browser**.
2. **Open** your **browser developer tools**, and **make sure** the **JavaScript console tab** is **open**.




127.0.0.1
Number guessing game
We have selected a random number between 1 and 100. See if you can guess it in 10 turns or fewer. We'll tell you if your guess was too high or too low.
Enter a guess: Hello
Submit guess
Type guessField into the console and the console shows you that the variable contains an <input> element.
Elements
Network
Sources
Timelines
Storage
Graphics
Audit
Console
Preserve Log
Emulate User Gesture
All
Evaluations
Errors
Warnings
Logs
Console cleared at 7:52:14 AM
guessField;
<input type="text" id="guessField" class="guessField"> = $1


127.0.0.1
Number guessing game
We have selected a random number between 1 and 100. See if you can guess it in 10 turns or fewer. We'll tell you if your guess was too high or too low.
Enter a guess: Hello
Submit guess
Search
Elements
Network
Console
Type guessField.value = 'Hello';
Preserve Log
Emulate User Gesture
All
Evaluations
Errors
Warnings
Logs
Console cleared at 7:52:14 AM
guessField;
<input type="text" id="guessField" class="guessField"> = $1
guessField.value = "Hello";
"Hello" = $2
guessField.value;
"Hello" = $2

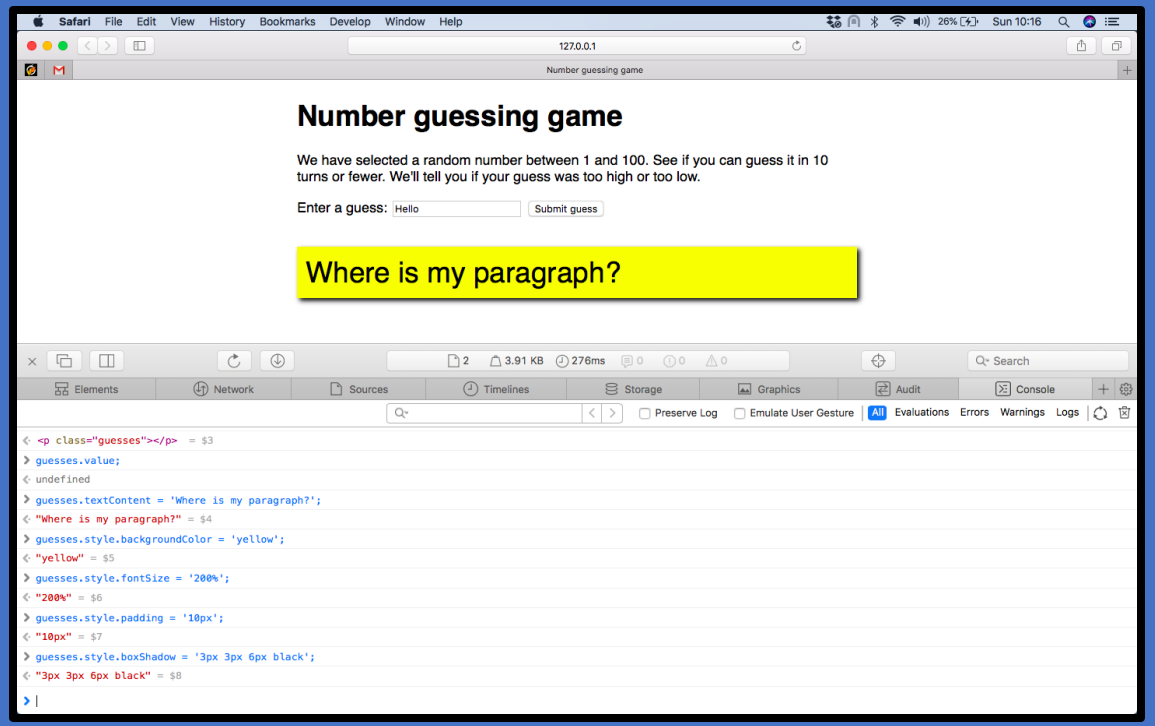

127.0.0.1
Number guessing game
We have selected a random number between 1 a
can guess it in 10 turns or fewer. We'll tell you if y
high or too low.
Enter a guess: Hello
Submit guess
1
2
3
4
5
Try typing guesses into the console and pressing return. The console shows you that the variable contains a element.
Search
Console
Preserve Log
Emulate User Gesture
All
Evaluations
Errors
Warnings
Logs
Console cleared at 7:55:34 AM
guesses;
 = $1


Safari File Edit View History Bookmarks Develop Window Help
127.0.0.1
Number guessing game
Number guessing game
guesses.textContent = 'Where is my paragraph?';
Enter a guess: Hello
Submit guess
Where is my paragraph?
guesses.style.backgroundColor = 'yellow';
guesses.style.fontSize = '200%';
guesses.style.padding = '10px';
guesses.style.boxShadow = '3px 3px 6px black';
Elements
Network
 = $3
guesses.value;
undefined
guesses.textContent = 'Where is my paragraph?';
guesses.style.backgroundColor = 'yellow';
"yellow" = $5
guesses.style.fontSize = '200%';
"200%" = $6
guesses.style.padding = '10px';
"10px" = $7
guesses.style.boxShadow = '3px 3px 6px black';
"3px 3px 6px black" = $8
1
2
3
4
5

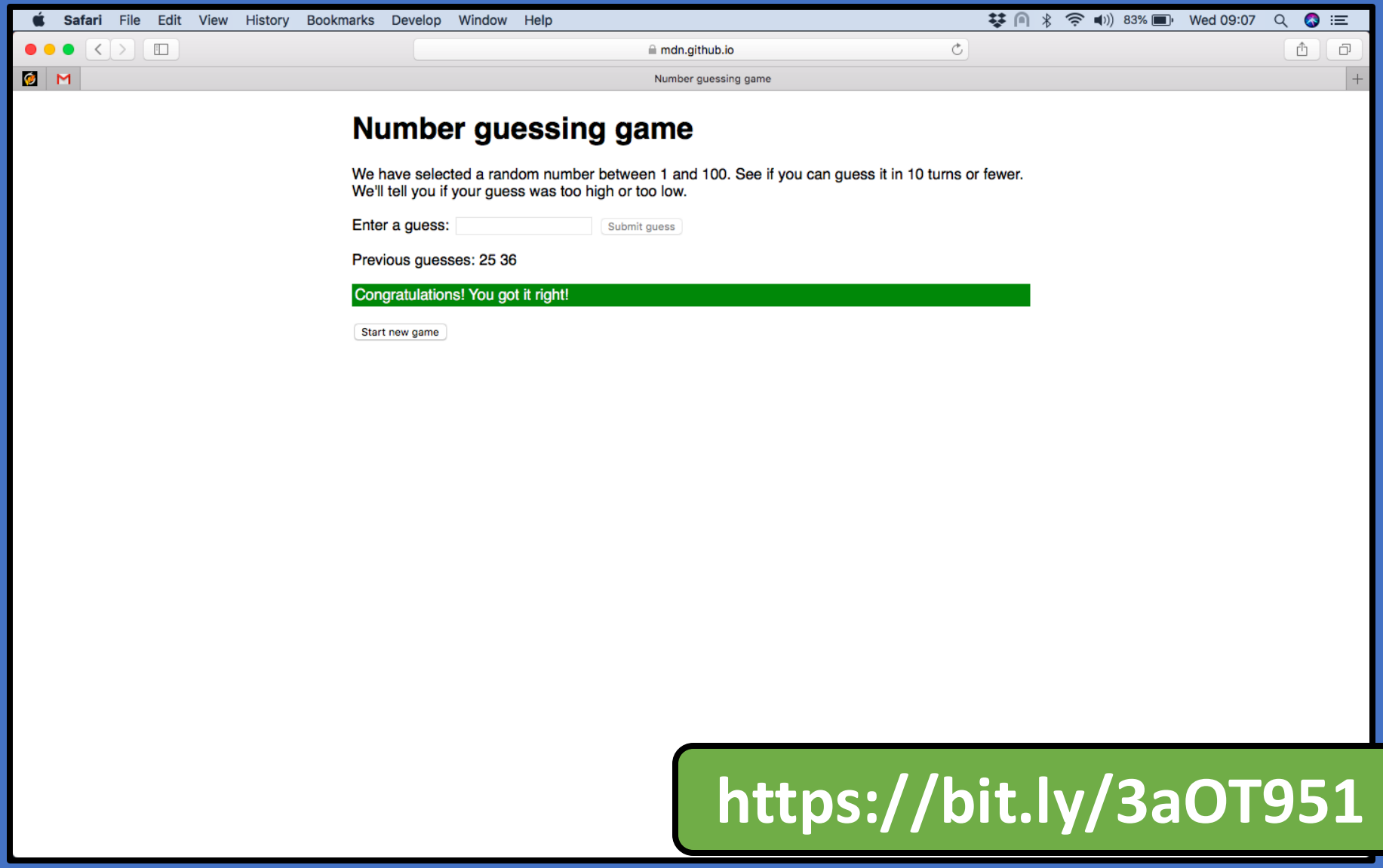
Playing with browser objects

- Every **element** on a **page** has a `style` **property.**
  - ✓This **property** contains an **object** whose **properties contain all** the **inline CSS styles applied** to that **element**.
  - ✓This **allows** us to **dynamically set new CSS styles** on **elements** using **JavaScript**.

```
1 guesses.style.backgroundColor = 'yellow';
2 guesses.style.fontSize = '200%';
3 guesses.style.padding = '10px';
4 guesses.style.boxShadow = '3px 3px 6px black';
```



Finished version



https://bit.ly/3aOT951

Questions?